

glmCH2_clean

Load and clean data

```
#load packages  
library(tidyverse) #data manipulation, visualization etc. (dplyr, ggplot2, tidyr)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
## v dplyr      1.1.4      v readr      2.1.5  
## v forcats    1.0.0      v stringr   1.5.1  
## v ggplot2    3.5.2      v tibble    3.3.0  
## v lubridate  1.9.4      v tidyr     1.3.1  
## v purrr      1.0.4  
## -- Conflicts ----- tidyverse_conflicts() --  
## x dplyr::filter() masks stats::filter()  
## x dplyr::lag()     masks stats::lag()  
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(DHARMA) #diagnostic tools for mixed regression models
```

```
## This is DHARMA 0.4.7. For overview type '?DHARMA'. For recent changes, type news(package = 'DHARMA')
```

```
library(glmmTMB) #fits generalized linear mixed models, including zero-inflated and overdispersed models  
library(car) #tools for regression diagnostics and hypothesis tests
```

```
## Loading required package: carData  
##  
## Attaching package: 'car'  
##  
## The following object is masked from 'package:dplyr':  
##  
##     recode  
##  
## The following object is masked from 'package:purrr':  
##  
##     some
```

```
library(emmeans) # for estimated marginal means (predictions)
```

```
## Welcome to emmeans.  
## Caution: You lose important information if you filter this package's results.  
## See '? untidy'
```

```

library(lubridate) #date <-> julian date

#read in data
data <- read.csv("06_cleandata/combined_plot_data_ch2.csv")

#change character columns to factors
surveys <- data %>%
  mutate(
    site_id = as.factor(site_id),
    round = as.factor(round),
    plot_id = as.factor(plot_id),
    bee_date = as.Date(bee_date),
    bee_season_type = as.factor(bee_season_type),
    area = as.factor(area),
    cover_type = as.factor(cover_type))

#update cover type names
levels(surveys$cover_type) <- c("bare", "herbaceous")

#add plot type column and make it a factor
surveys <- surveys %>%
  mutate(plot_type = if_else(plot_id == "d", "ditch", "field"))
surveys <- surveys %>%
  mutate(
    plot_type = as.factor(plot_type))

#scale continuous predictors
surveys$julian.date_scaled <- as.numeric(scale(surveys$julian.date))
surveys$bare_perc_scaled <- as.numeric(scale(surveys$bare_perc))
surveys$grass_perc_scaled <- as.numeric(scale(surveys$grass_perc))
surveys$forb_perc_scaled <- as.numeric(scale(surveys$forb_perc))
surveys$grass_forb_perc_scaled <- as.numeric(scale(surveys$grass_forb_perc))
surveys$grass_h_scaled <- as.numeric(scale(surveys$grass_h))
surveys$forb_h_scaled <- as.numeric(scale(surveys$forb_h))
surveys$floral_perc_scaled <- as.numeric(scale(surveys$floral_perc))
surveys$grass_forb_floral_perc_scaled <- as.numeric(scale(surveys$grass_forb_floral_perc))
surveys$bombus_floral_perc_scaled <- as.numeric(scale(surveys$bombus_floral_perc))
surveys$flood_perc_scaled <- as.numeric(scale(surveys$flood_perc))
surveys$dry_root_perc_scaled <- as.numeric(scale(surveys$dry_root_perc))
surveys$bryo_perc_scaled <- as.numeric(scale(surveys$bryo_perc))
surveys$bare_flood_dry_bryo_perc_scaled <- as.numeric(scale(surveys$bare_flood_dry_bryo_perc))
surveys$floral_richness_scaled <- as.numeric(scale(surveys$floral_richness))
surveys$bombus_floral_richness_scaled <- as.numeric(scale(surveys$bombus_floral_richness))
surveys$bombus_floral_abun_scaled <- as.numeric(scale(surveys$bombus_floral_abun))
surveys$nest_searching_queens_scaled <- as.numeric(scale(surveys$nest_searching_queens))
surveys$workers_scaled <- as.numeric(scale(surveys$workers))
surveys$forage_vaco_scaled <- as.numeric(scale(surveys$forage_vaco))
surveys$num_burrows_scaled <- as.numeric(scale(surveys$num_burrows))

```

Choosing which vegetation variables to include as predictors

1. Use PCA to determine which vegetation variables to include in my models. I don't want to include all of the predictors, since some will be correlated with each other and therefore dilute their effects of the response variables. I am using PCA so I can pick one variable that drives each of the main 2 components, and use these as the predictors in my model.

```
pca_res <- prcomp(surveys[, c("bare_perc_scaled",
                              "grass_forb_perc_scaled",
                              "grass_h_scaled",
                              "forb_h_scaled",
                              "bombus_floral_perc_scaled",
                              "flood_perc_scaled",
                              "dry_root_perc_scaled",
                              "bombus_floral_richness_scaled",
                              "bombus_floral_abun_scaled")], scale. = TRUE)

summary(pca_res)
```

```
## Importance of components:
##              PC1    PC2    PC3    PC4    PC5    PC6    PC7
## Standard deviation  1.8037 1.2128 1.0888 0.94326 0.90070 0.82536 0.54028
## Proportion of Variance 0.3615 0.1634 0.1317 0.09886 0.09014 0.07569 0.03243
## Cumulative Proportion 0.3615 0.5249 0.6566 0.75551 0.84565 0.92135 0.95378
##              PC8    PC9
## Standard deviation  0.5187 0.38329
## Proportion of Variance 0.0299 0.01632
## Cumulative Proportion 0.9837 1.00000
```

```
#see which variable drives each of the principle components
print(pca_res$rotation)
```

```
##              PC1    PC2    PC3    PC4
## bare_perc_scaled  0.24100694 0.51132677 0.1480093 -0.506337915
## grass_forb_perc_scaled -0.42303293 -0.22390273 -0.3007926 -0.006568266
## grass_h_scaled -0.49812518 -0.11060633 -0.0470433 -0.149976271
## forb_h_scaled -0.48569527 -0.08315431 0.1134057 -0.240186492
## bombus_floral_perc_scaled -0.18883655 0.40010344 -0.1653710 0.651841412
## flood_perc_scaled 0.12306241 -0.07874923 -0.6378149 0.019972489
## dry_root_perc_scaled 0.02157359 -0.40268963 0.6227065 0.317142224
## bombus_floral_richness_scaled -0.43081001 0.26087490 0.1437940 -0.233788645
## bombus_floral_abun_scaled -0.20509491 0.52226932 0.1730539 0.287860892
##              PC5    PC6    PC7    PC8
## bare_perc_scaled 0.06538925 0.370121563 0.49842251 0.10038801
## grass_forb_perc_scaled -0.27778537 -0.153122311 0.54910763 0.48948701
## grass_h_scaled 0.02851587 0.203518533 0.20705746 -0.38586034
## forb_h_scaled 0.15759445 0.172355381 -0.08782852 -0.41865475
## bombus_floral_perc_scaled -0.13014055 0.571974934 -0.03009873 0.03525875
## flood_perc_scaled 0.74841113 0.058657287 0.08301863 0.01827233
## dry_root_perc_scaled 0.42325486 0.207845797 0.28968027 0.20646515
## bombus_floral_richness_scaled 0.27097277 0.003296897 -0.48095986 0.58651301
## bombus_floral_abun_scaled 0.25156245 -0.628196361 0.27646719 -0.19483913
##              PC9
```

```
## bare_perc_scaled          -0.049064111
## grass_forb_perc_scaled    -0.196630307
## grass_h_scaled            0.693496680
## forb_h_scaled             -0.670154480
## bombus_floral_perc_scaled -0.075975027
## flood_perc_scaled         -0.025721397
## dry_root_perc_scaled      0.012084553
## bombus_floral_richness_scaled 0.149185034
## bombus_floral_abun_scaled -0.006968346
```

- PCA Results:

- PC1: grass_h_scaled (-0.50), forb_h_scaled (-0.48), grass_forb_perc_scaled (-0.42), bombus_floral_richness_scaled (-0.43), bombus_floral_abun_scaled (-0.22)
- PC2: bare_perc_scaled (0.51), bombus_floral_abun_scaled (0.52), bombus_floral_perc_scaled (0.40), dry_root_perc_scaled (-0.40)

- Based on these PCA results and my research question, I am selecting to include the following predictors in my model:

- grass_forb_perc_scaled (loads strongly on PCA1)
- bombus_floral_abun_scaled (loads strongly on PCA1 & 2)

3. Check pairwise correlations between this reduced set of variables

- correlation among all the pairs are all near zero -> predictors are all sufficiently independent! ($R \ll 0.7$)

```
cor(surveys[, c("grass_forb_perc_scaled", "bombus_floral_abun_scaled")])
```

```
##               grass_forb_perc_scaled bombus_floral_abun_scaled
## grass_forb_perc_scaled             1.00000000             0.07455522
## bombus_floral_abun_scaled          0.07455522             1.00000000
```

Comparing field and ditch plots at grassy and bare farms.

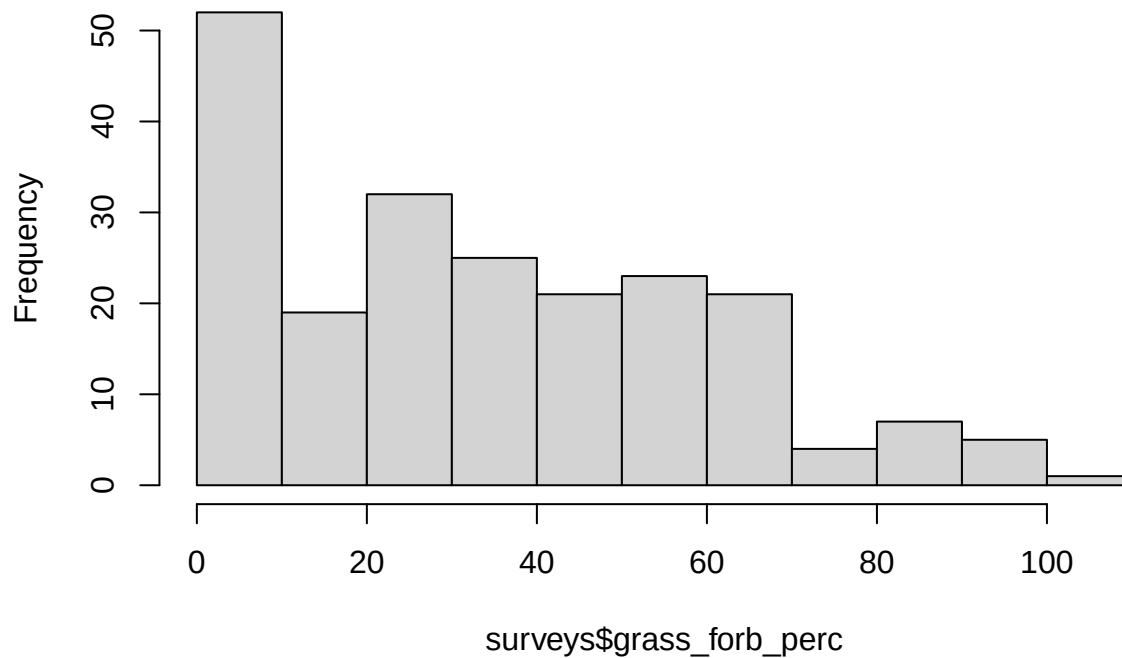
Vegetative cover

Steps

Check raw distribution of grass/forb %

```
hist(surveys$grass_forb_perc)
```

Histogram of surveys\$grass_forb_perc



```
summary(surveys$grass_forb_perc)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   \n##      0.00  10.20   31.88   34.48  53.14  102.92
```

Run tweedie model w full random effect structure

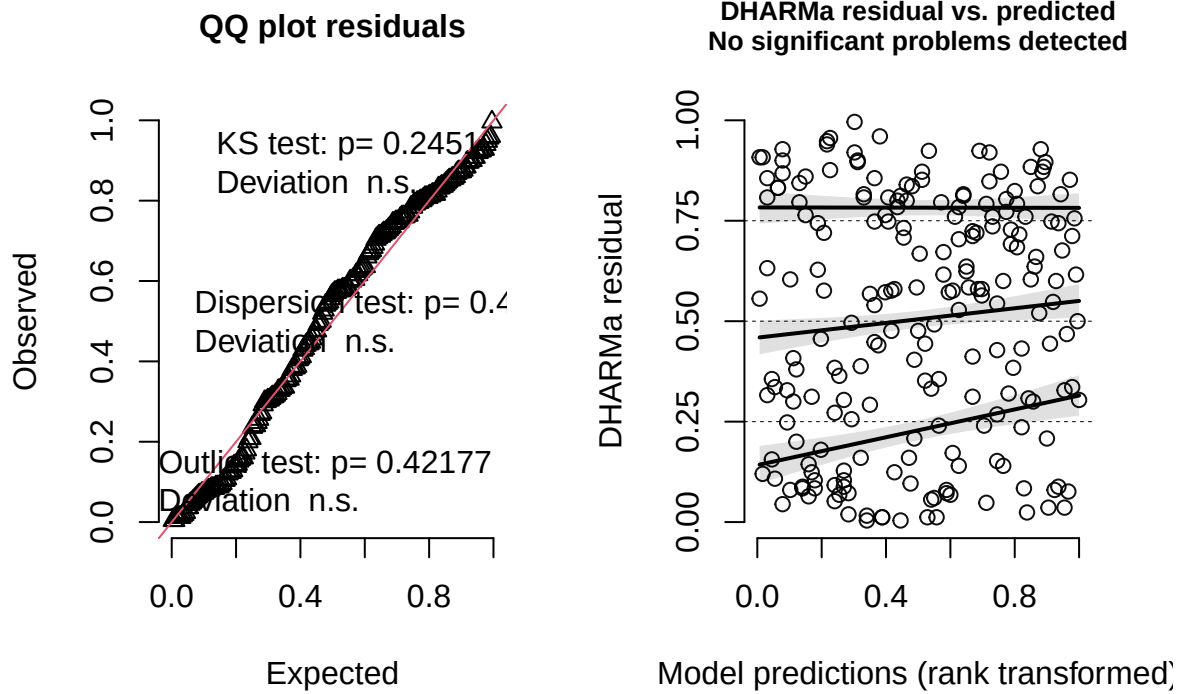
```
ditchVSfield_veg_tweedie <- glmmTMB(\n  grass_forb_perc ~ cover_type*plot_type +\n    julian.date_scaled +\n    (1|area/site_id/plot_id),\n  family = tweedie(link = "log"),\n  data = surveys\n)
```

Check diagnostics

```
#simulate residuals
```

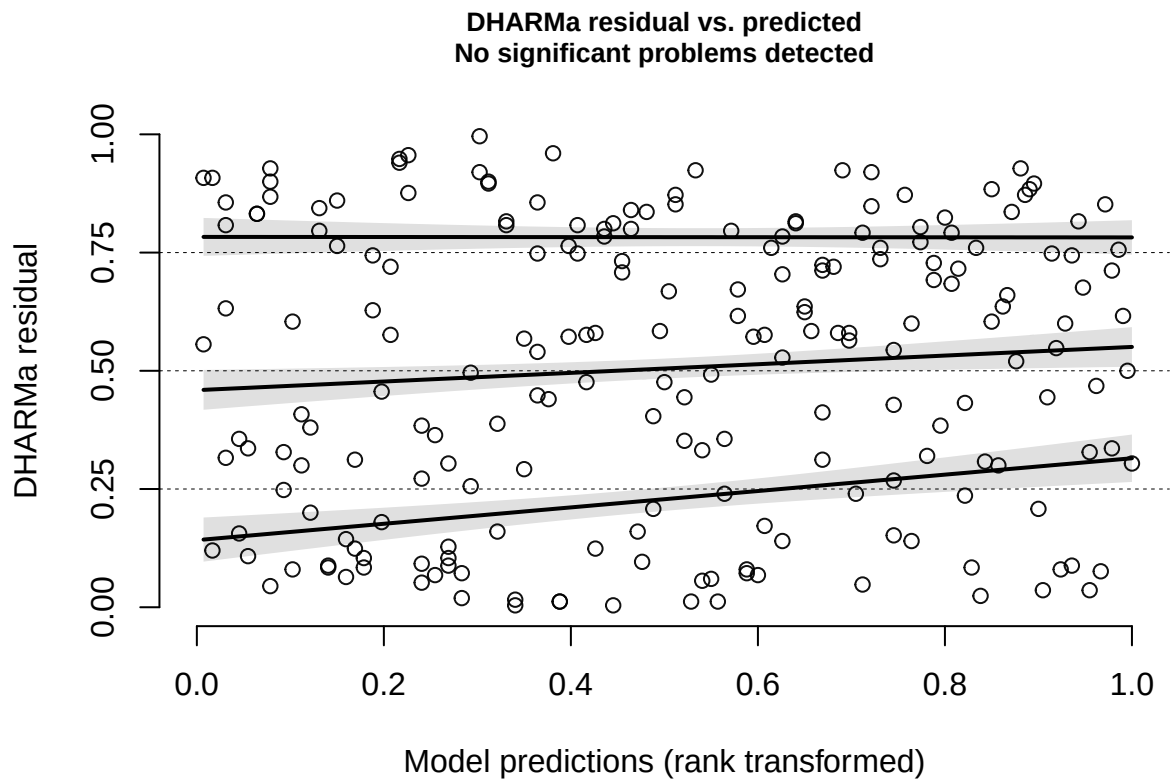
```
residuals_ditchVSfield_veg_tweedie <- simulateResiduals(fittedModel = ditchVSfield_veg_tweedie, plot = '')
```

DHARMA residual



##looks good!

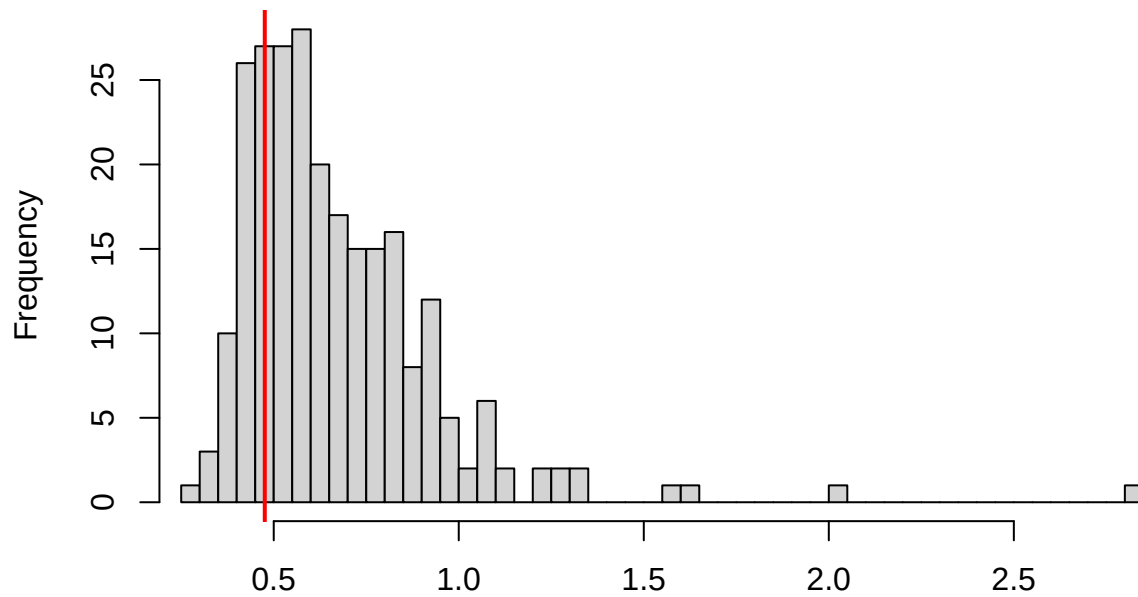
```
testQuantiles(residuals_ditchVSfield_veg_tweedie) #ns
```



```
##  
## Test for location of quantiles via qgam  
##  
## data: res  
## p-value = 0.2339  
## alternative hypothesis: both
```

```
testDispersion(residuals_ditchVSfield_veg_tweedie) #n.s
```

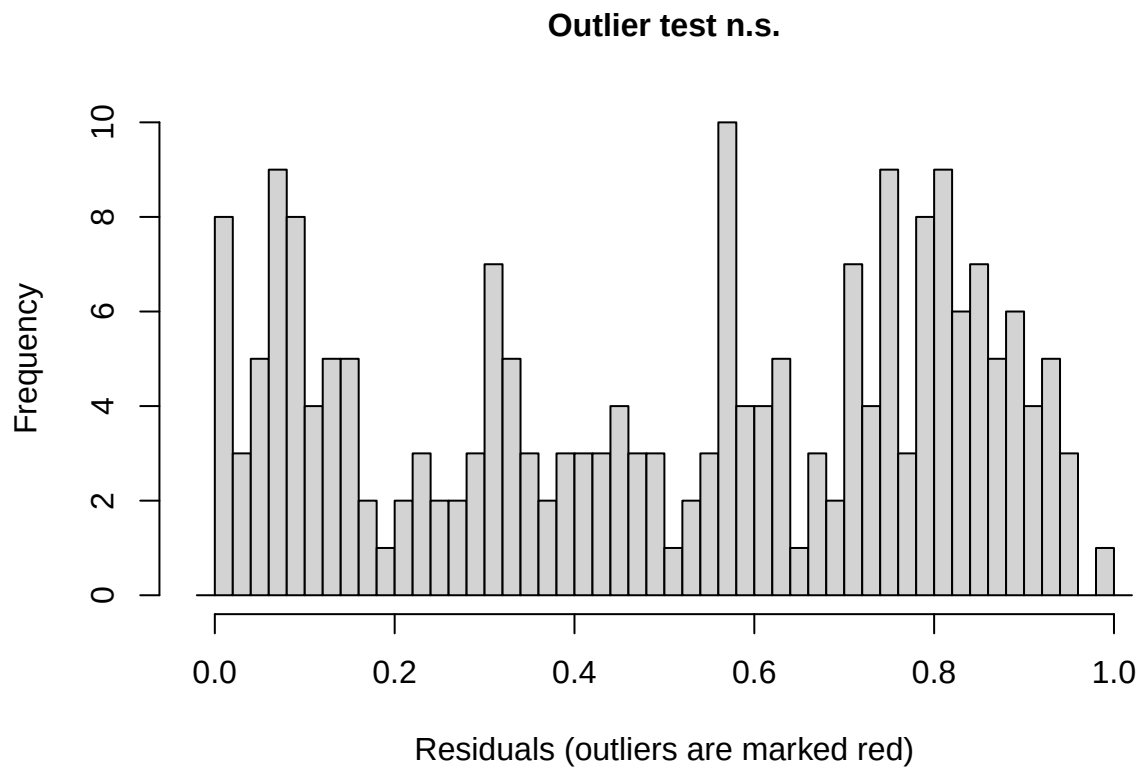
DHARMA nonparametric dispersion test via sd of residuals fitted vs. simulated



Simulated values, red line = fitted model. p-value (two.sided) = 0.4

```
##
## DHARMA nonparametric dispersion test via sd of residuals fitted vs.
## simulated
##
## data: simulationOutput
## dispersion = 0.70802, p-value = 0.4
## alternative hypothesis: two.sided
```

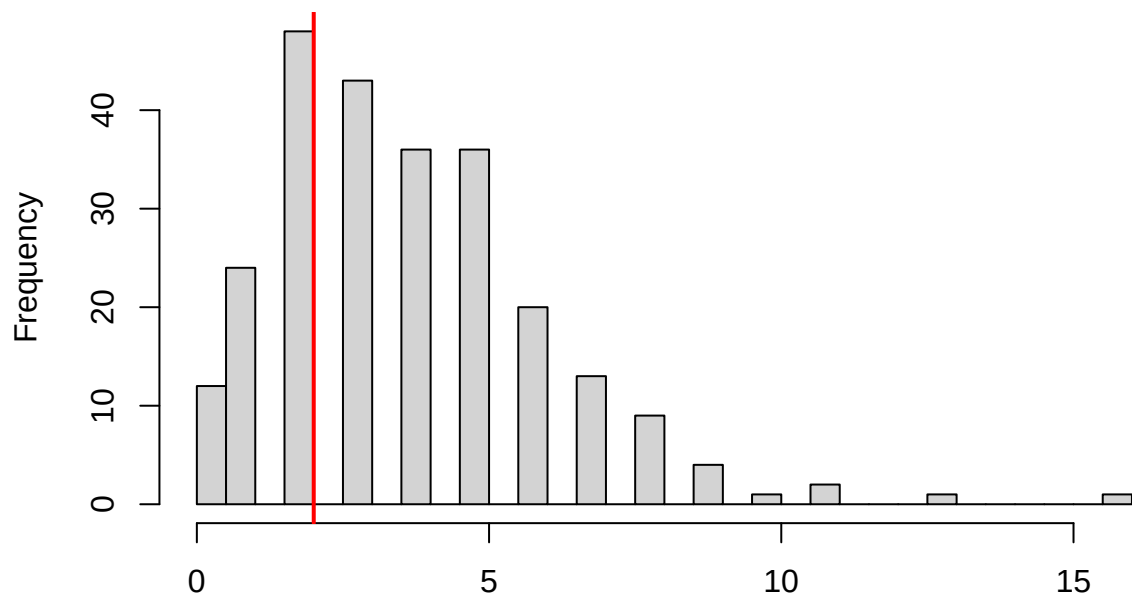
```
testOutliers(residuals_ditchVSfield_veg_tweedie) #ns
```



```
##
## DHARMA outlier test based on exact binomial test with approximate
## expectations
##
## data: residuals_ditchVSfield_veg_tweedie
## outliers at both margin(s) = 0, observations = 210, p-value = 0.4218
## alternative hypothesis: true probability of success is not equal to 0.007968127
## 95 percent confidence interval:
##  0.00000000 0.01741271
## sample estimates:
## frequency of outliers (expected: 0.00796812749003984 )
## 0
```

```
testZeroInflation(residuals_ditchVSfield_veg_tweedie) #ns
```

DHARMa zero-inflation test via comparison to expected zeros with simulation under H0 = fitted model



Simulated values, red line = fitted model. p-value (two.sided) = 0.672

```
##
## DHARMa zero-inflation test via comparison to expected zeros with
## simulation under H0 = fitted model
##
## data: simulationOutput
## ratioObsSim = 0.52466, p-value = 0.672
## alternative hypothesis: two.sided
```

Print output

```
summary(ditchVSfield_veg_tweedie)
```

```
## Family: tweedie ( log )
## Formula:
## grass_forb_perc ~ cover_type * plot_type + julian.date_scaled +
## (1 | area/site_id/plot_id)
## Data: surveys
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    1772.6   1806.1   -876.3    1752.6      200
##
## Random effects:
##
## Conditional model:
## Groups          Name      Variance Std.Dev.
```

```
## plot_id:site_id:area (Intercept) 1.170e-09 3.421e-05
## site_id:area (Intercept) 5.920e-02 2.433e-01
## area (Intercept) 6.945e-02 2.635e-01
## Number of obs: 210, groups:
## plot_id:site_id:area, 36; site_id:area, 12; area, 6
##
## Dispersion parameter for tweedie family (): 2.47
##
## Conditional model:
##
## Estimate Std. Error z value Pr(>|z|)
## (Intercept) 3.65829 0.17004 21.514 < 2e-16 ***
## cover_typeherbaceous 0.32693 0.18255 1.791 0.0733 .
## plot_typefield -1.09816 0.12108 -9.069 < 2e-16 ***
## julian.date_scaled -0.04614 0.03752 -1.230 0.2188
## cover_typeherbaceous:plot_typefield 0.75980 0.15604 4.869 1.12e-06 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Find effect of cover type within each plot type

```
library(emmeans)

pairs(emmeans(ditchVSfield_veg_tweedie, ~ cover_type | plot_type))
```

```
## plot_type = ditch:
## contrast estimate SE df z.ratio p.value
## bare - herbaceous -0.327 0.183 Inf -1.791 0.0733
##
## plot_type = field:
## contrast estimate SE df z.ratio p.value
## bare - herbaceous -1.087 0.176 Inf -6.184 <.0001
##
## Results are given on the log (not the response) scale.
```

Find effect of plot type within each cover type

```
pairs(emmeans(ditchVSfield_veg_tweedie, ~ plot_type | cover_type))
```

```
## cover_type = bare:
## contrast estimate SE df z.ratio p.value
## ditch - field 1.098 0.1210 Inf 9.069 <.0001
##
## cover_type = herbaceous:
## contrast estimate SE df z.ratio p.value
## ditch - field 0.338 0.0983 Inf 3.441 0.0006
##
## Results are given on the log (not the response) scale.
```

Plots

Cover type x plot type

```

#create new data grid
veg_data<- expand.grid(
  cover_type = unique(surveys$cover_type),
  plot_type = unique(surveys$plot_type),
  julian.date_scaled = mean(surveys$julian.date_scaled, na.rm = TRUE)
)

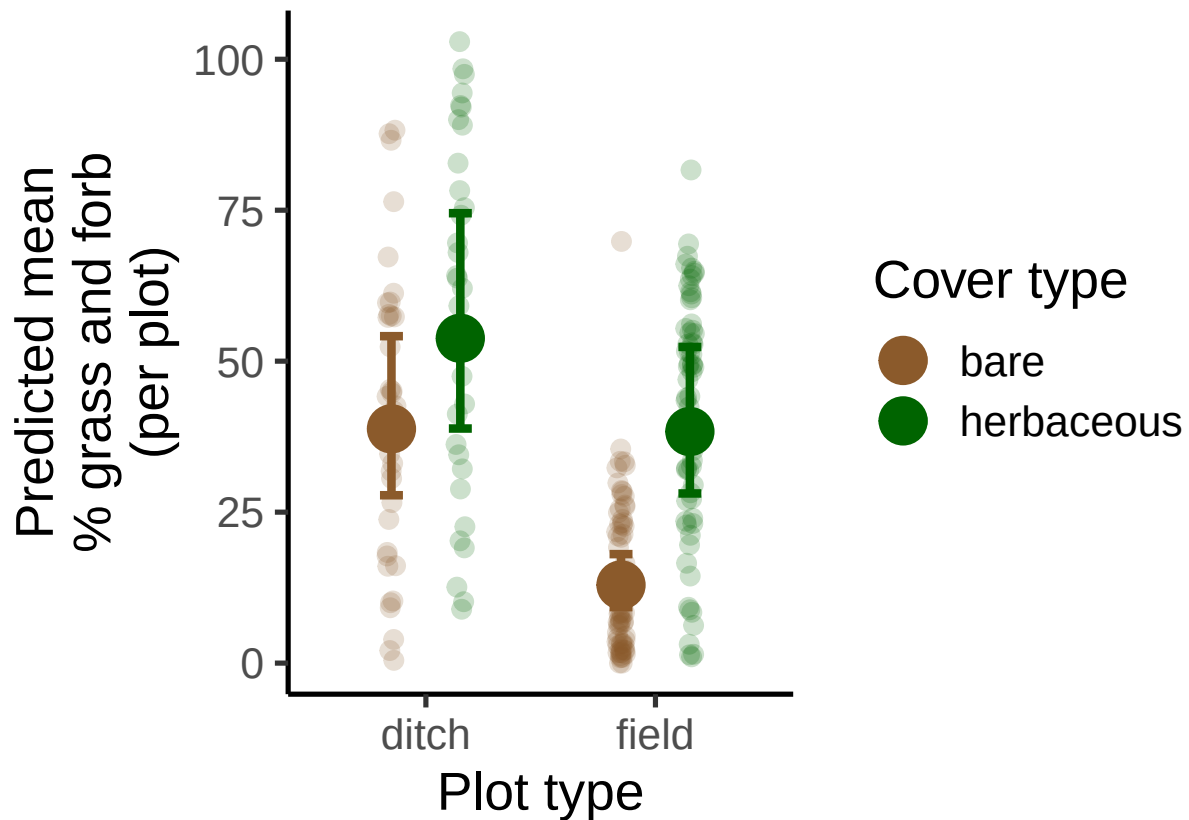
# Get predicted probabilities and standard errors
pred_veg <- predict(ditchVSfield_veg_tweedie,
  newdata = veg_data,
  se.fit = TRUE,
  re.form = NA,
  type = "link")

# Combine predictions with data frame
pred_veg_df <- cbind(veg_data,
  fit_log = pred_veg$fit,
  se_log = pred_veg$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_veg_df <- pred_veg_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

# Plot conditional predictions
ggplot(pred_veg_df, aes(x = plot_type, y = fit, color = cover_type)) +
  geom_jitter(
    data = surveys,
    aes(x = plot_type, y = grass_forb_perc, color = cover_type),
    position = position_jitterdodge(
      jitter.width = 0.08,
      dodge.width = 0.6),
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size=8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth=1)
# Use scale_color_manual to define specific colors
scale_color_manual(values = c("herbaceous" = "darkgreen", "bare" = "tan4")) +
labs(x = "Plot type",
  y = "Predicted mean\n% grass and forb\n(per plot)",
  color = "Cover type") +
theme_classic(base_size = 20)

```



Julian date

```
date_seq <- seq(
  from = min(surveys$julian.date_scaled, na.rm = TRUE),
  to = max(surveys$julian.date_scaled, na.rm = TRUE),
  length.out = 50
)

#create new data grid
veg_data_date<- expand.grid(
  cover_type = "herbaceous",
  plot_type = "ditch",
  julian.date_scaled = date_seq
)

# Get predicted probabilities and standard errors
pred_veg_date <- predict(ditchVSfield_veg_tweedie,
  newdata = veg_data_date,
  se.fit = TRUE,
  re.form = NA,
  type = "link")

# Combine predictions with data frame
pred_veg_date_df <- cbind(veg_data_date,
  fit_log = pred_veg_date$fit,
  se_log = pred_veg_date$se.fit)
```

```

#calculate predictions and confidence intervals on response scale (not log scale)
pred_veg_date_df <- pred_veg_date_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

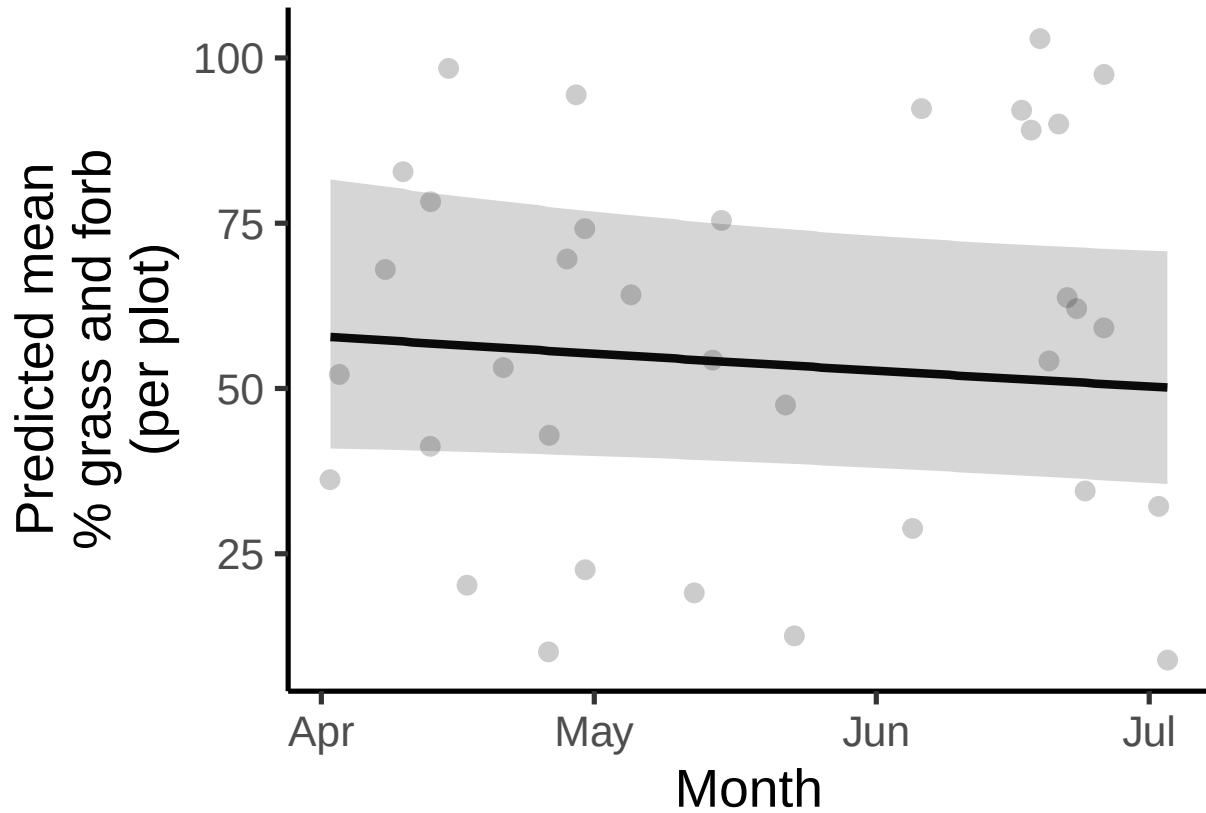
#convert x-axis to actual dates
## Get scaling parameters from original data
jd_mean <- mean(surveys$julian.date, na.rm = TRUE)
jd_sd <- sd(surveys$julian.date, na.rm = TRUE)

## Convert scaled values back to dates
pred_veg_date_df <- pred_veg_date_df %>%
  mutate(
    # Reverse the (x - mean) / sd formula
    julian_orig = (julian.date_scaled * jd_sd) + jd_mean,
    # Convert Day of Year to an actual Date object (using 2024 as a reference year)
    date_formatted = as.Date(round(julian_orig) - 1, origin = "2024-01-01")
  )

#add date formatted to surveys data frame
surveys <- surveys %>%
  mutate(
    date_formatted = as.Date(
      round(julian.date) - 1,
      origin = "2024-01-01"
    )
  )

#Plot
ggplot(pred_veg_date_df, aes(x = date_formatted, y = fit)) +
  geom_jitter(
    data = subset(surveys, cover_type == "herbaceous" & plot_type == "ditch"),
    aes(x = date_formatted, y = grass_forb_perc),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2, color=NA) +
  scale_x_date(date_breaks = "1 month", date_labels = "%b") + # Format x-axis to show Month names (e.g.
  labs(x = "Month",
    y = "Predicted mean\n% grass and forb\n(per plot)") +
  theme_classic(base_size = 20)

```



Findings

Cover type

- Significantly more veg at grassy sites than bare both along the ditch ($p=0.037$) and within the field ($p<0.0001$)
- Significant interaction between plot type and cover type, difference in veg between grassy and bare is much greater in the field than along the ditch ($p=0.0004$)
- Significantly less veg within the field than along the ditch ($p<0.0001$)

Date

- % veg decreases over time, though not significant ($p=0.118$)

Floral

Steps

Run model

```
#Load packages needed for CLM
library(ordinal)
```

```
##
## Attaching package: 'ordinal'

## The following object is masked from 'package:dplyr':
##
##     slice
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'

## The following object is masked from 'package:dplyr':
##
##     select
```

```
surveys$bombus_floral_abun <- factor(surveys$bombus_floral_abun, ordered = TRUE)

floral_full <- clmm(bombus_floral_abun ~ cover_type*plot_type +
                    julian.date_scaled*plot_type +
                    (1|area/site_id/plot_id),
                    link = "logit",
                    data = surveys)
```

Print results

```
summary(floral_full)
```

```
## Cumulative Link Mixed Model fitted with the Laplace approximation
##
## formula: bombus_floral_abun ~ cover_type * plot_type + julian.date_scaled *
##          plot_type + (1 | area/site_id/plot_id)
## data:    surveys
##
## link threshold nobs logLik AIC      niter      max.grad cond.H
## logit flexible  210  -317.10 658.19 2136(4286) 4.66e-04 1.5e+02
##
## Random effects:
## Groups              Name          Variance Std.Dev.
## plot_id:(site_id:area) (Intercept) 0.00000  0.0000
## site_id:area          (Intercept) 0.07232  0.2689
## area                  (Intercept) 0.00000  0.0000
## Number of groups: plot_id:(site_id:area) 36, site_id:area 12, area 6
##
## Coefficients:
##                                     Estimate Std. Error z value Pr(>|z|)
## cover_typeherbaceous                0.2142     0.4353   0.492 0.622720
```

```
## plot_typefield          0.3737      0.3673      1.017 0.309073
## julian.date_scaled      0.6590      0.2082      3.165 0.001549 **
## cover_typeherbaceous:plot_typefield -0.1895      0.5143     -0.369 0.712498
## plot_typefield:julian.date_scaled   -0.9281      0.2651     -3.501 0.000464 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Threshold coefficients:
##      Estimate Std. Error z value
## 0|1  -1.2764      0.3368  -3.790
## 1|2  -0.7679      0.3251  -2.362
## 2|3   0.1277      0.3187   0.401
## 3|4   1.2033      0.3314   3.631
```

Plots

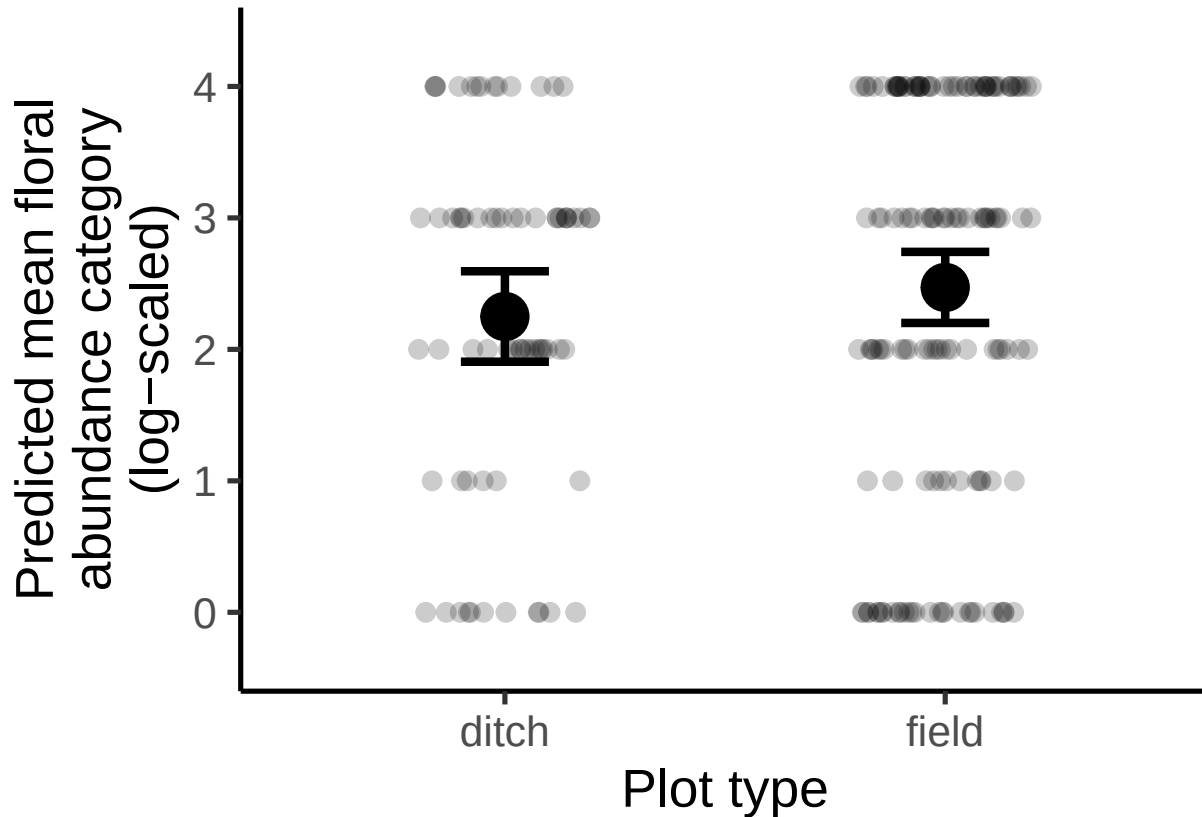
Plot type

```
# Get predicted probabilities per category with CIs
pred_floral_plot <- emmeans(
  floral_full,
  ~ plot_type,
  mode = "mean.class" ## The expected floral abundance category (mean ordinal score)
)
```

NOTE: Results may be misleading due to involvement in interactions

```
# Convert to dataframe
pred_floral_plot_df <- as.data.frame(pred_floral_plot)

#plot
ggplot(pred_floral_plot_df, aes(y = mean.class, x = plot_type)) +
  geom_jitter(
    data = surveys,
    aes(x = plot_type, y = bombus_floral_abun),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = asymp.LCL, ymax = asymp.UCL), width = 0.2, linewidth = 1.5, position = position_dodge(0.6)) +
  labs(
    x = "Plot type",
    y = "Predicted mean floral abundance category (log-scaled)"
  ) +
  theme_classic(base_size = 20)
```



Cover type

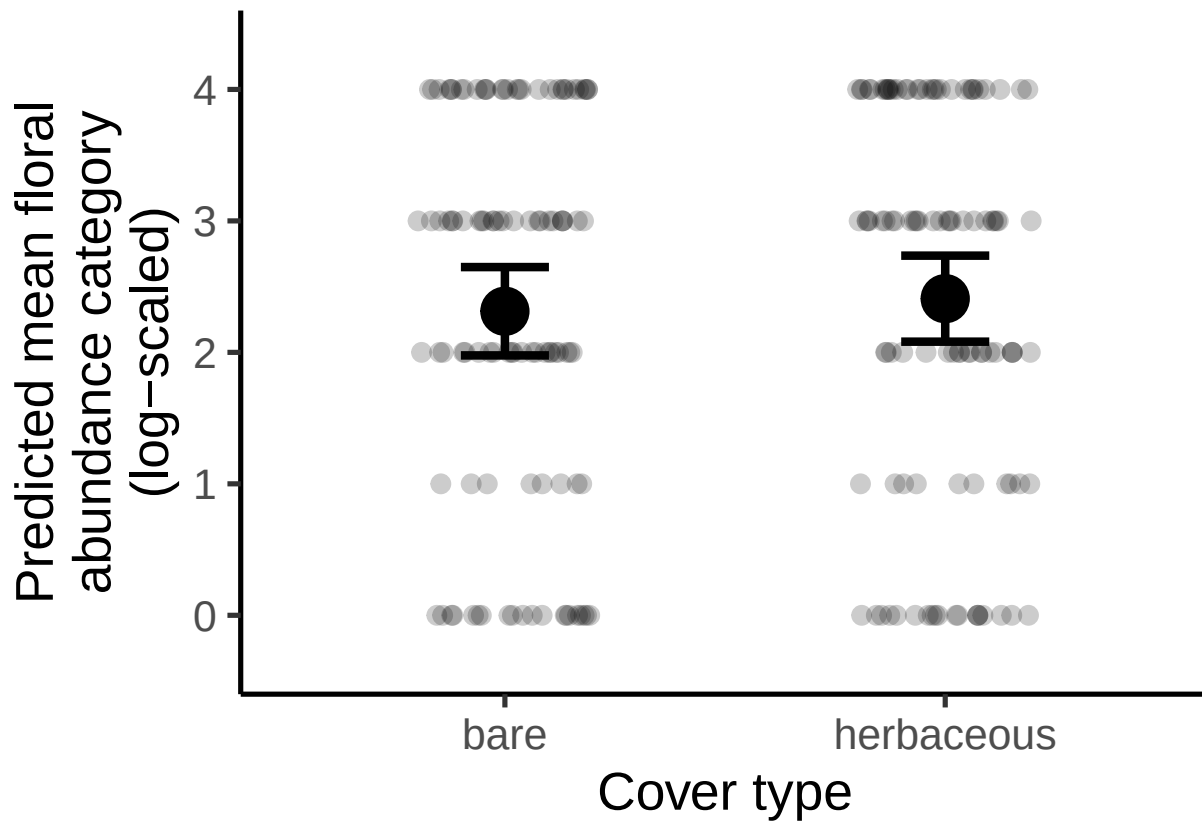
```
# Get predicted probabilities per category with CIs
pred_floral_cover <- emmeans(
  floral_full,
  ~ cover_type,
  mode = "mean.class" ## The expected floral abundance category (mean ordinal score)
)
```

NOTE: Results may be misleading due to involvement in interactions

```
# Convert to dataframe
pred_floral_cover_df <- as.data.frame(pred_floral_cover)

#plot
ggplot(pred_floral_cover_df, aes(x = cover_type, y = mean.class)) +
  geom_jitter(
    data = surveys,
    aes(x = cover_type, y = bombus_floral_abun),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = asymp.LCL, ymax = asymp.UCL), width = 0.2, linewidth = 1.5, position = position_dodge(0.6))
```

```
labs(
  x = "Cover type",
  y = "Predicted mean floral abundance category (log-scaled)"
) +
theme_classic(base_size = 20)
```



Season x plot type

```
# Sequence of Julian dates
date_seq <- seq(
  from = min(surveys$julian.date_scaled, na.rm = TRUE),
  to = max(surveys$julian.date_scaled, na.rm = TRUE),
  length.out = 200
)

# Get predicted probabilities per category with CIs
pred_floral_date <- emmeans(
  floral_full,
  ~ julian.date_scaled * plot_type,
  at = list(julian.date_scaled = date_seq),
  mode = "mean.class" ## The expected floral abundance category (mean ordinal score)
)

# Convert to dataframe
pred_floral_date_df <- as.data.frame(pred_floral_date)
```

```

#convert x-axis to actual dates
## Convert scaled values back to dates
pred_floral_date_df <- pred_floral_date_df %>%
  mutate(
    # Reverse the (x - mean) / sd formula
    julian_orig = (julian.date_scaled * jd_sd) + jd_mean,
    # Convert Day of Year to an actual Date object (using 2024 as a reference year)
    date_formatted = as.Date(round(julian_orig) - 1, origin = "2024-01-01")
  )

min(pred_floral_date_df$date_formatted, na.rm = TRUE)

```

```
## [1] "2024-04-02"
```

```
max(pred_floral_date_df$date_formatted, na.rm = TRUE)
```

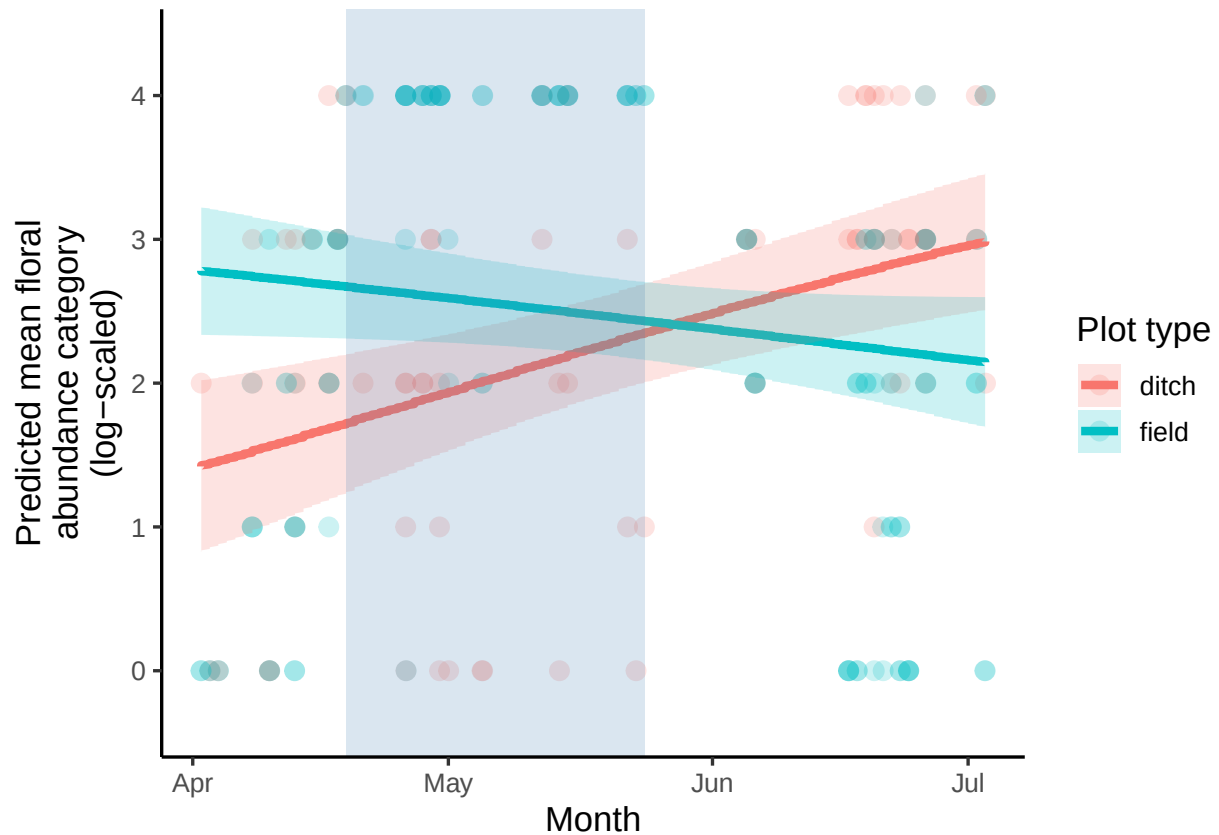
```
## [1] "2024-07-03"
```

```

#set peak bloom start and end dates
bloom_start <- as.Date("2024-04-19")
bloom_end <- as.Date("2024-05-24")

#plot
ggplot(pred_floral_date_df, aes(x = date_formatted, y = mean.class, color = plot_type)) +
  geom_jitter(
    data = surveys,
    aes(x = date_formatted, y = bombus_floral_abun, color = plot_type),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = asymp.LCL, ymax = asymp.UCL, fill = plot_type, group = plot_type), alpha = 0.2)
  scale_x_date(date_breaks = "1 month", date_labels = "%b") + # Format x-axis to show Month names (e.g.
  annotate("rect", xmin = bloom_start, xmax = bloom_end, ymin = -Inf, ymax = Inf,
    alpha = .2, fill = "steelblue")+
  labs(
    x = "Month",
    y = "Predicted mean floral\nabundance category\n(log-scaled)",
    color = "Plot type",
    fill = "Plot type"
  ) +
  theme_classic(base_size = 13)

```



Findings

Cover type

- No significant difference in floral abundance between ditch and field at beginning of the study ($t=1.01$)
- No difference in floral abundance between grass and bare sites ($t=0.531$).
- No significant interaction between plot type and cover type ($t=-0.360$)

Date

- Significant effect of julian date ($t=3.112$)
- Significant interaction between julian date and plot type ($t=-3.444$).
 - Ditches get more flowers over time (+0.64 per unit of time).
 - Fields get fewer flowers over time (-0.26 per unit of time)

Rodents

Steps

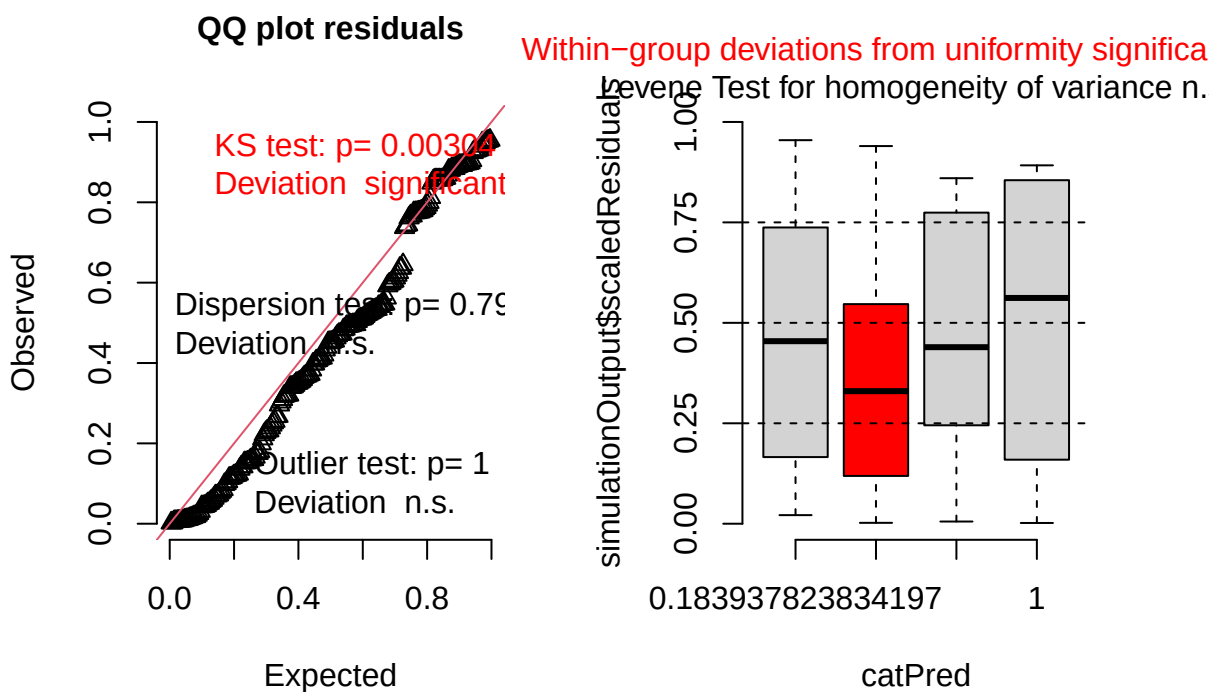
- 1) Run poisson model (no plot random effect since only one observation per plot)

```
rodent_veg_full <- glmmTMB(num_burrows ~ cover_type*plot_type +
  (1|area/site_id),
  family = poisson,
  data = surveys)
```

2) Check diagnostics

```
residuals_rodent_veg_full <- simulateResiduals(fittedModel = rodent_veg_full, plot = TRUE)
```

DHARMA residual



##KS test shows significant deviation

3) Try negative binomial and check diagnostics.

- diagnostic plots look good!

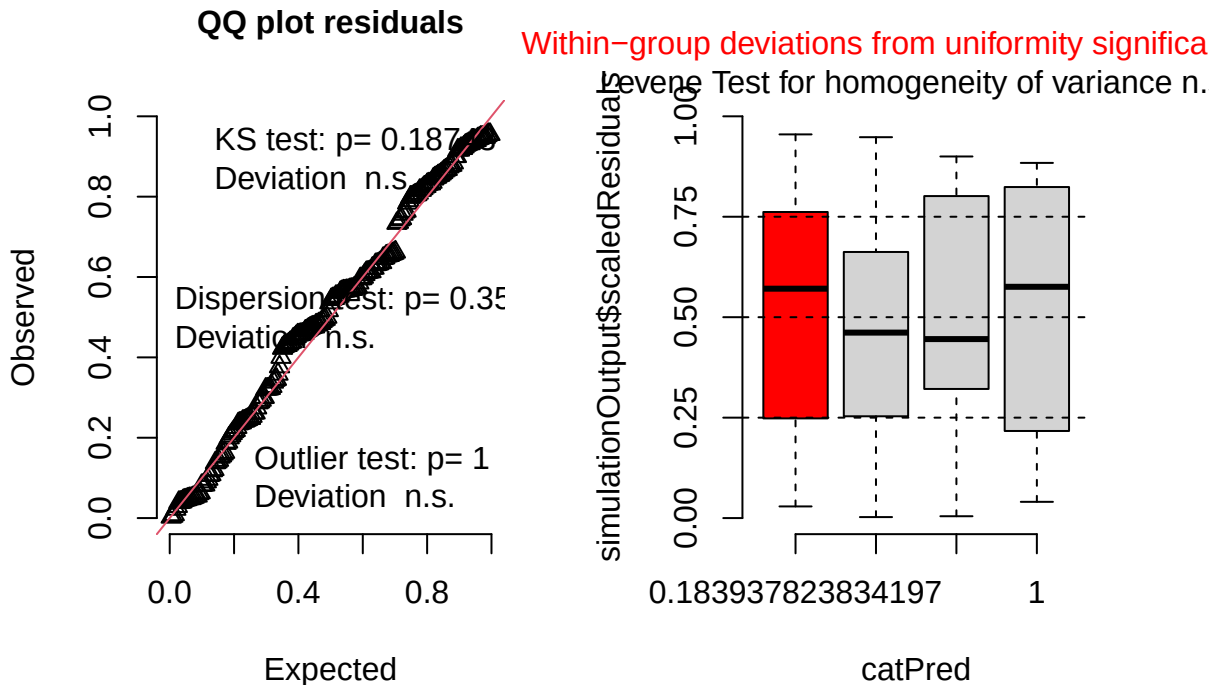
```
rodent_veg_full_nbin <- glmmTMB(num_burrows ~ cover_type*plot_type +
  (1|area/site_id),
  family = nbinom2,
  data = surveys)
```

#check diagnostics

#simulate residuals

```
residuals_rodent_veg_full_nbin <- simulateResiduals(fittedModel = rodent_veg_full_nbin, plot = TRUE)
```

DHARMA residual



##looks good

4. Plot results.

```
summary(rodent_veg_full_nbin)
```

```
## Family: nbinom2 ( log )
## Formula:          num_burrows ~ cover_type * plot_type + (1 | area/site_id)
## Data: surveys
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    1485.8    1509.2    -735.9    1471.8     203
##
## Random effects:
##
## Conditional model:
##   Groups      Name      Variance Std.Dev.
## site_id:area (Intercept) 1.69477  1.302
## area         (Intercept) 0.07181  0.268
## Number of obs: 210, groups:  site_id:area, 12; area, 6
##
## Dispersion parameter for nbinom2 family (): 1.43
##
## Conditional model:
##                                     Estimate Std. Error z value Pr(>|z|)
```

```
## (Intercept)                3.1649      0.5692    5.560 2.7e-08 ***
## cover_typeherbaceous        0.3146      0.7885    0.399 0.68992
## plot_typefield              -1.9925      0.2314   -8.609 < 2e-16 ***
## cover_typeherbaceous:plot_typefield  0.9873      0.3141    3.143 0.00167 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Find effect of cover type in each plot type

```
pairs(emmeans(rodent_veg_full_nbin, ~ cover_type | plot_type))
```

```
## plot_type = ditch:
## contrast      estimate      SE  df z.ratio p.value
## bare - herbaceous  -0.315 0.789 Inf  -0.399 0.6899
##
## plot_type = field:
## contrast      estimate      SE  df z.ratio p.value
## bare - herbaceous  -1.302 0.774 Inf  -1.682 0.0926
##
## Results are given on the log (not the response) scale.
```

Find effect of plot type in each cover type

```
pairs(emmeans(rodent_veg_full_nbin, ~ plot_type | cover_type))
```

```
## cover_type = bare:
## contrast      estimate      SE  df z.ratio p.value
## ditch - field    1.99 0.231 Inf    8.609 <.0001
##
## cover_type = herbaceous:
## contrast      estimate      SE  df z.ratio p.value
## ditch - field    1.01 0.212 Inf    4.738 <.0001
##
## Results are given on the log (not the response) scale.
```

Plots

Plot type

```
#create new data grid
rodent_data_plot<- expand.grid(
  cover_type = "herbaceous",
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE)
)

# Get predicted probabilities and standard errors
pred_rodent_plot <- predict(rodent_veg_full_nbin,
  newdata = rodent_data_plot,
  se.fit = TRUE,
  re.form = NA,
```

```

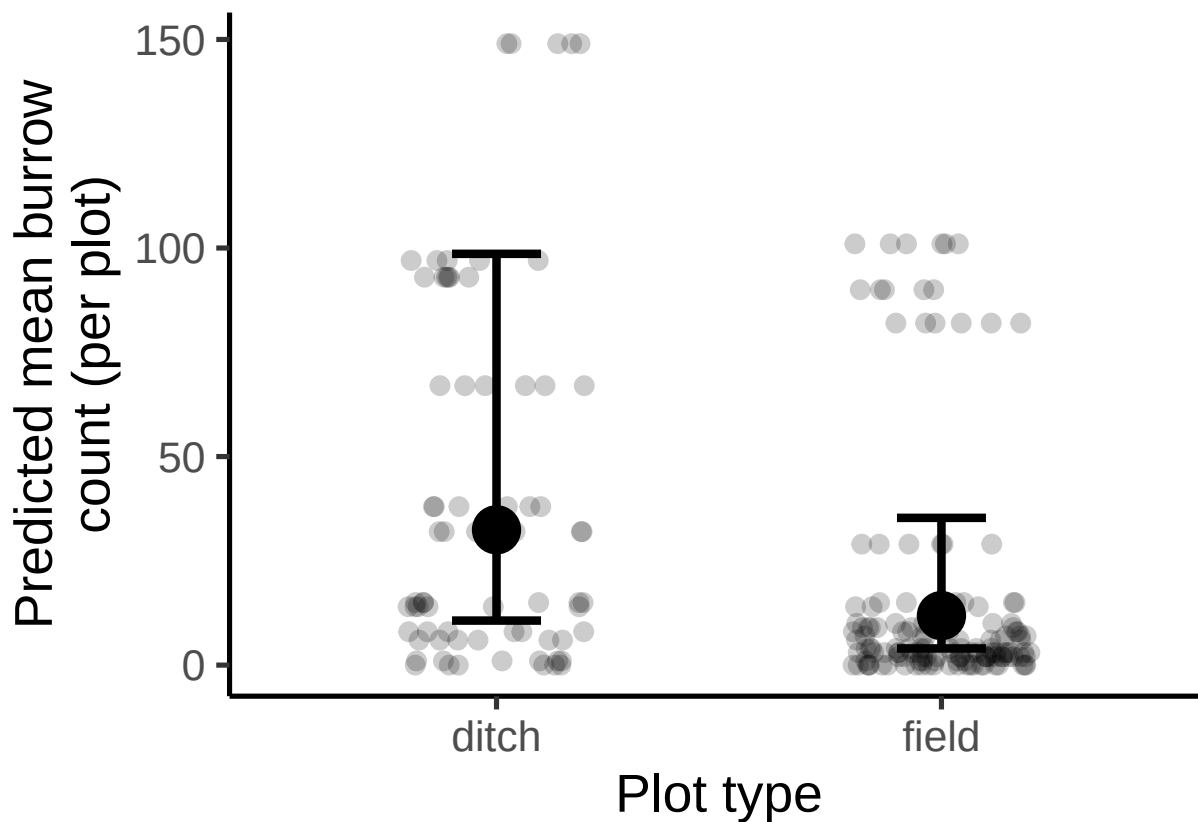
        type = "link")

# Combine predictions with data frame
pred_rodent_plot_df <- cbind(rodent_data_plot,
                             fit_log = pred_rodent_plot$fit,
                             se_log = pred_rodent_plot$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_rodent_plot_df <- pred_rodent_plot_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

# Plot conditional predictions
ggplot(pred_rodent_plot_df, aes(x = plot_type, y = fit)) +
  geom_jitter(
    data = surveys,
    aes(x = plot_type, y = num_burrows),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size=8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Plot type",
       y = "Predicted mean burrow\ncount (per plot)") +
  theme_classic(base_size = 20)

```



cover type x plot type

```
#create new data grid
rodent_data_veg<- expand.grid(
  cover_type = unique(surveys$cover_type),
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE)
)

# Get predicted probabilities and standard errors
pred_rodent_veg <- predict(rodent_veg_full_nbin,
  newdata = rodent_data_veg,
  se.fit = TRUE,
  re.form = NA,
  type = "link")

# Combine predictions with data frame
pred_rodent_veg_df <- cbind(rodent_data_veg,
  fit_log = pred_rodent_veg$fit,
  se_log = pred_rodent_veg$se.fit)

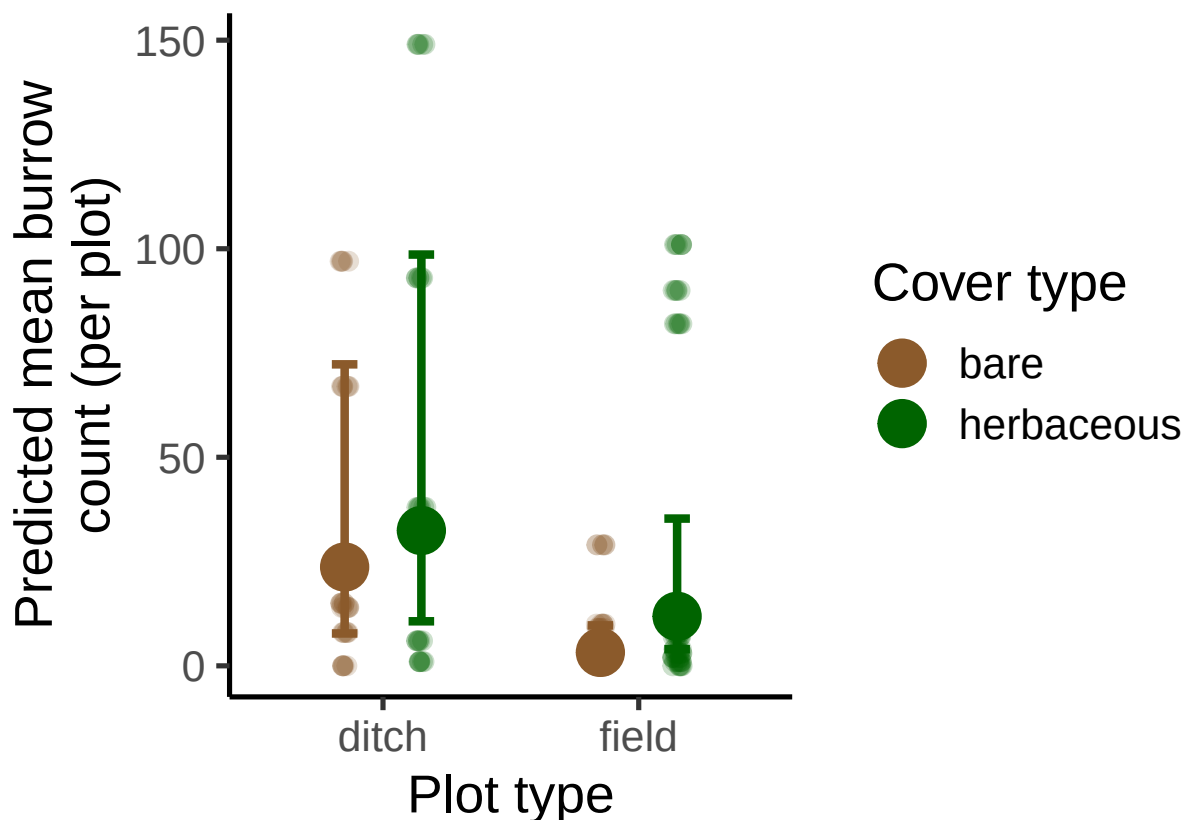
#calculate predictions and confidence intervals on response scale (not log scale)
pred_rodent_veg_df <- pred_rodent_veg_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
```

```

    upper = exp(fit_log + 1.96 * se_log)
  )

# Plot conditional predictions
ggplot(pred_rodent_veg_df, aes(x = plot_type, y = fit, color = cover_type)) +
  geom_jitter(
    data = surveys,
    aes(x = plot_type, y = num_burrows, color = cover_type),
    position = position_jitterdodge(
      jitter.width = 0.08,
      dodge.width = 0.6),
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size=8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth=1) +
  labs(x = "Plot type",
       y = "Predicted mean burrow\ncount (per plot)",
       color = "Cover type") +
  scale_color_manual(values = c("herbaceous" = "darkgreen", "bare" = "tan4")) +
  theme_classic(base_size = 20)

```



Findings

- Significant interaction between cover type and plot type ($p < 0.0001$)

- Marginal effect of cover type on rodent burrow abundance within the field only ($p=0.0735$)
- No effect of cover type on burrow abundance along the ditch (probably because less difference in vegetation) ($p=0.826$)
- Significantly more rodents along the ditch than within the field ($p<0.0001$)

NSQ

Plot model with interaction between vegetation variables and plot type to investigate whether the effects of vegetation cover and floral abundance on queen abundance/presence change depending on plot type (ditch vs field).

Steps

1. Run poisson with full random effect structure.

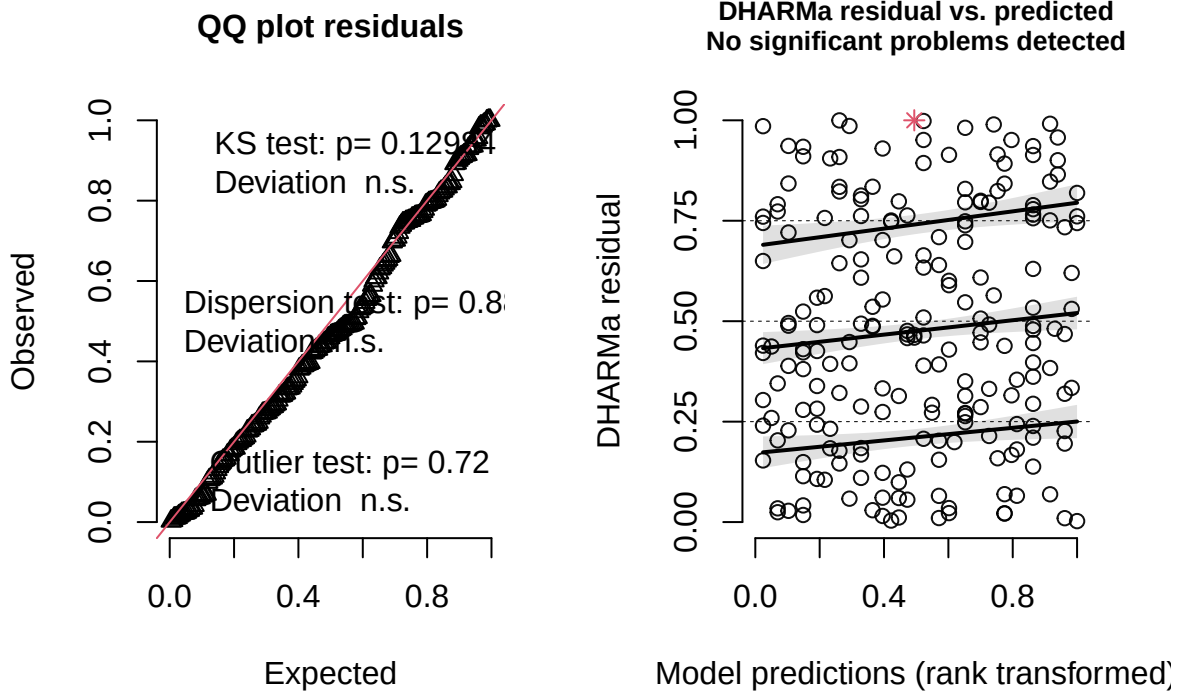
```
nsq_glm_veg_full <- glmmTMB(nest_searching_queens ~ cover_type*plot_type+
                           bombus_floral_abun_scaled*plot_type +
                           bee_season_type +
                           (1|area/site_id/plot_id),
                           family = poisson,
                           data = surveys)
```

3. Check diagnostics of model

- look good

```
#simulate residuals
residuals_nsq_glm_veg_full <- simulateResiduals(fittedModel = nsq_glm_veg_full, plot = TRUE)
```

DHARMA residual



4. Plot results

```
summary(nsq_glm_veg_full)
```

```
## Family: poisson ( log )
## Formula:
## nest_searching_queens ~ cover_type * plot_type + bombus_floral_abun_scaled *
##   plot_type + bee_season_type + (1 | area/site_id/plot_id)
## Data: surveys
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    296.0    329.4   -138.0    276.0     200
##
## Random effects:
##
## Conditional model:
## Groups          Name      Variance Std.Dev.
## plot_id:site_id:area (Intercept) 5.279e-01 7.266e-01
## site_id:area        (Intercept) 8.842e-11 9.403e-06
## area                (Intercept) 7.411e-09 8.609e-05
## Number of obs: 210, groups:
## plot_id:site_id:area, 36; site_id:area, 12; area, 6
##
## Conditional model:
##                                     Estimate Std. Error z value Pr(>|z|)
```

```
## (Intercept) -0.4918 0.4455 -1.104 0.2696
## cover_typeherbaceous 0.9176 0.5477 1.676 0.0938
## plot_typefield -1.3544 0.5978 -2.266 0.0235
## bombus_floral_abun_scaled -0.2206 0.1391 -1.586 0.1128
## bee_season_typeworker -2.4055 0.4002 -6.011 1.85e-09
## cover_typeherbaceous:plot_typefield 0.2768 0.7494 0.369 0.7118
## plot_typefield:bombus_floral_abun_scaled 0.4774 0.2327 2.051 0.0403
##
## (Intercept)
## cover_typeherbaceous .
## plot_typefield *
## bombus_floral_abun_scaled
## bee_season_typeworker ***
## cover_typeherbaceous:plot_typefield
## plot_typefield:bombus_floral_abun_scaled *
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Effect of floral abundance within each plot type

```
#calculate the slope of floral abundance for each plot type
eff_slopes <- emtrends(nsq_glm_veg_full, ~ plot_type, var = "bombus_floral_abun_scaled")

#test if those slopes are significantly different from zero
summary(eff_slopes, infer = c(TRUE, TRUE))
```

```
## plot_type bombus_floral_abun_scaled.trend SE df asymp.LCL asymp.UCL
## ditch -0.221 0.139 Inf -0.493 0.0521
## field 0.257 0.185 Inf -0.107 0.6201
## z.ratio p.value
## -1.586 0.1128
## 1.385 0.1660
##
## Results are averaged over the levels of: cover_type, bee_season_type
## Confidence level used: 0.95
```

Plots

Plot type

```
# Create newdata
nsq_data_plottype <- expand.grid(
  cover_type = "herbaceous",
  plot_type = factor(c("ditch", "field"), levels = levels(surveys$plot_type))
)
nsq_data_plottype$bombus_floral_abun_scaled <- mean(surveys$bombus_floral_abun_scaled)
nsq_data_plottype$bee_season_type <- "nestsearching"

# Get predicted zero probabilities and standard errors
pred_nsq_plottype_cond <- predict(nsq_glm_veg_full,
  newdata = nsq_data_plottype,
```

```

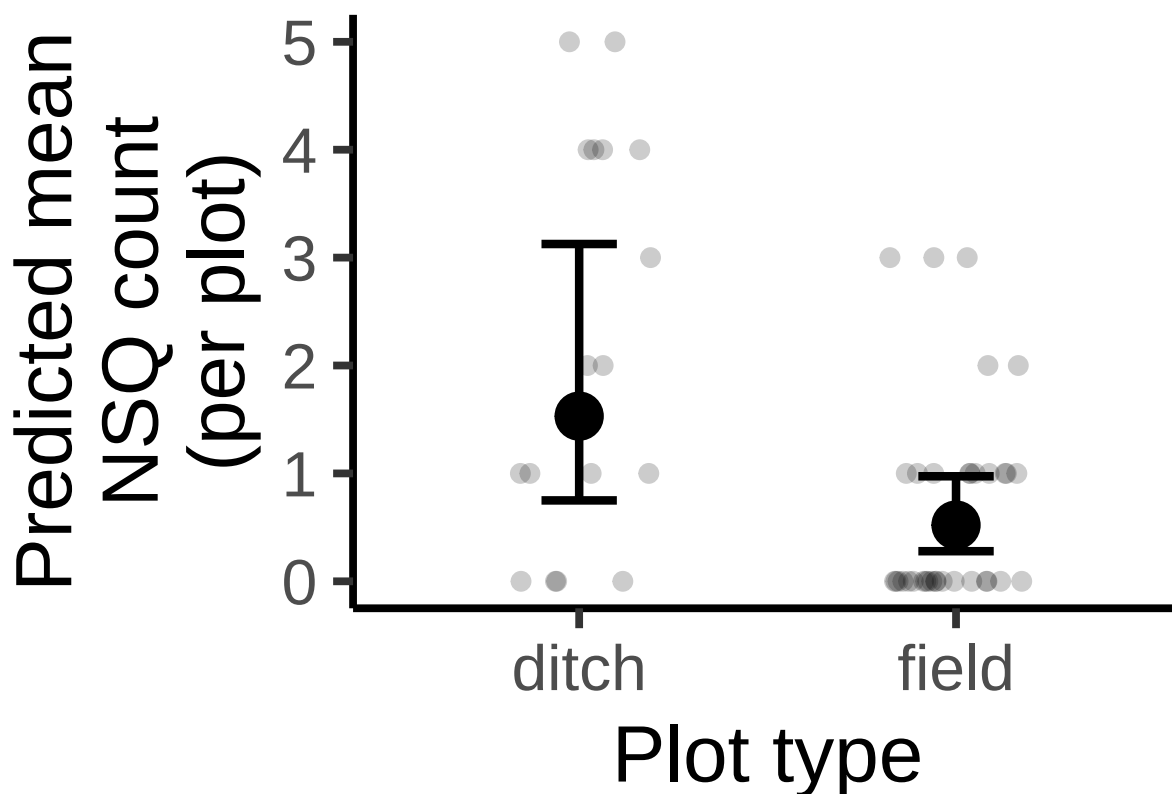
        type = "link",
        se.fit = TRUE,
        re.form = NA
      )

# Combine predictions with data frame
pred_nsq_plottype_df <- cbind(nsq_data_plottype,
                             cond_fit_log = pred_nsq_plottype_cond$fit,
                             cond_se_log = pred_nsq_plottype_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_plottype_df <- pred_nsq_plottype_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

#Plot
ggplot(pred_nsq_plottype_df, aes(x = plot_type, y = cond_fit)) +
  geom_jitter(
    data = subset(surveys, cover_type == "herbaceous" & bee_season_type == "nestsearching"),
    aes(x = plot_type, y = nest_searching_queens),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
  labs(x = "Plot type",
       y = "Predicted mean\nNSQ count\n(per plot)") +
  theme_classic(base_size = 30)

```



Cover type

```
# Create newdata
nsq_data_covertime <- expand.grid(
  cover_type = factor(c("herbaceous", "bare"), levels = levels(surveys$cover_type)),
  plot_type = "ditch"
)
nsq_data_covertime$bombus_floral_abun_scaled <- mean(surveys$bombus_floral_abun_scaled)
nsq_data_covertime$bee_season_type <- "nestsearching"

# Get predicted zero probabilities and standard errors
pred_nsq_covertime_cond <- predict(nsq_glm_veg_full,
  newdata = nsq_data_covertime,
  type = "link",
  se.fit = TRUE,
  re.form = NA)

# Combine predictions with data frame
pred_nsq_covertime_df <- cbind(nsq_data_covertime,
  cond_fit_log = pred_nsq_covertime_cond$fit,
  cond_se_log = pred_nsq_covertime_cond$se.fit)

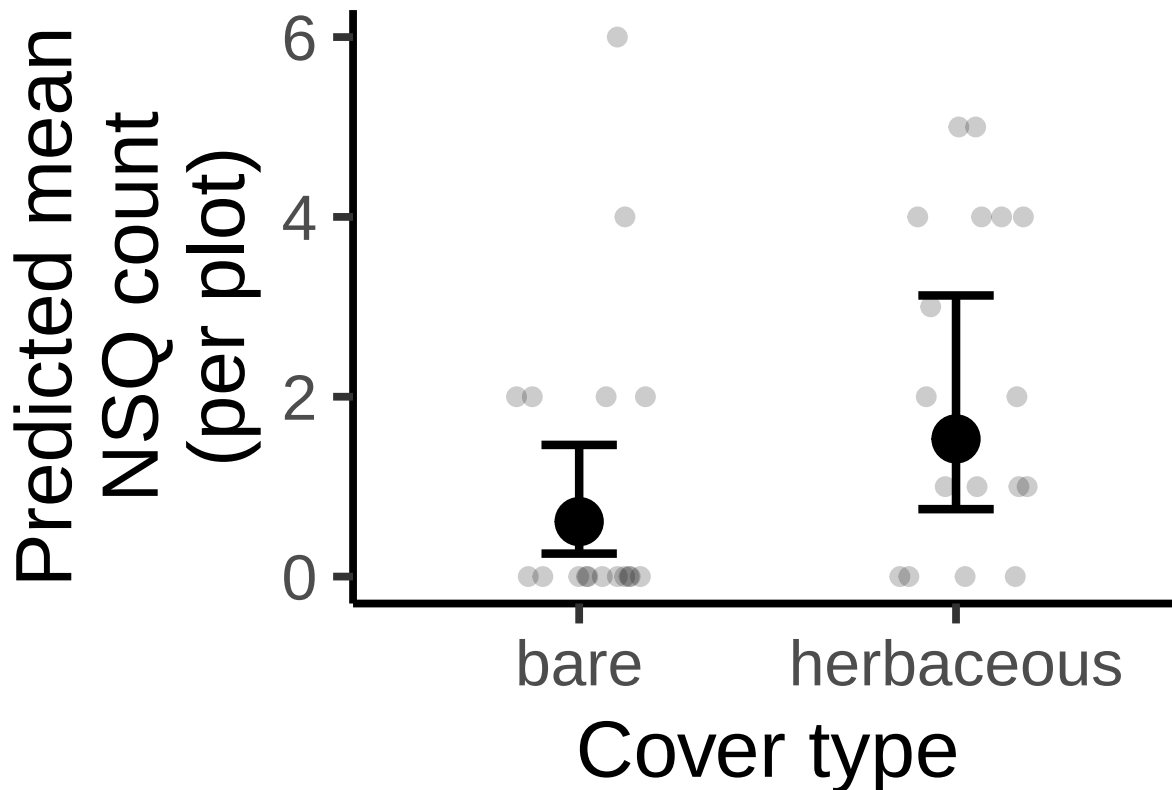
# calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_covertime_df <- pred_nsq_covertime_df %>%
  mutate(
```

```

cond_fit = exp(cond_fit_log),
cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
)

#Plot
ggplot(pred_nsq_covertime_df, aes(x = cover_type, y = cond_fit)) +
  geom_jitter(
    data = subset(surveys, plot_type == "ditch" & bee_season_type == "nestsearching"),
    aes(x = cover_type, y = nest_searching_queens),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
    labs(x = "Cover type",
      y = "Predicted mean\nNSQ count\n(per plot)") +
    theme_classic(base_size = 30)

```



Floral abundance

```

# Define a sequence of values for grass_forb_perc_scaled across its observed range
floral_seq <- seq(
  from = min(surveys$bombus_floral_abun_scaled, na.rm = TRUE),

```

```

    to = max(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
    length.out = 50
  )

  #create new data grid
  nsq_data_floral<- expand.grid(
    cover_type = "herbaceous",
    plot_type = "ditch",
    bombus_floral_abun_scaled = floral_seq,
    bee_season_type = "nestsearching"
  )

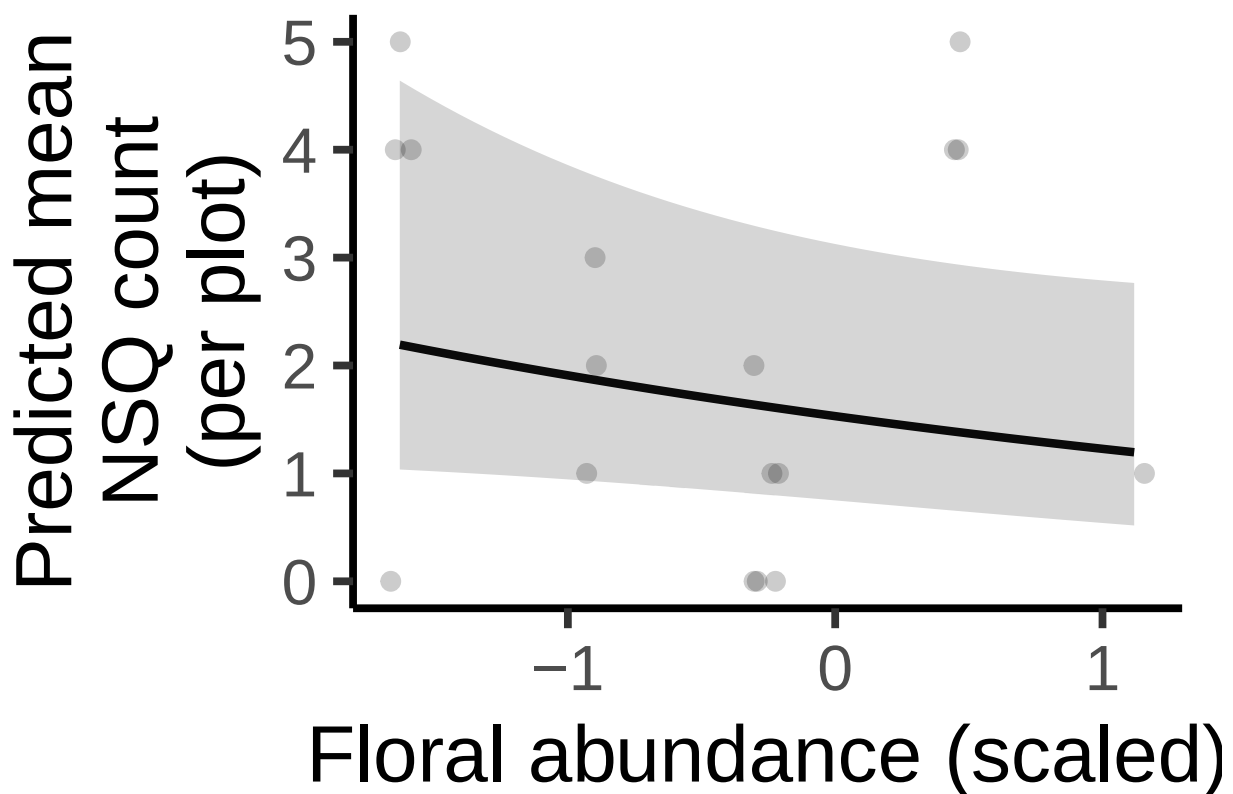
  # Get predicted conditional probabilities and standard errors
  pred_nsq_floral_cond <- predict(nsq_glm_veg_full,
                                newdata = nsq_data_floral,
                                type = "link",
                                se.fit = TRUE,
                                re.form = NA)

  # Combine predictions with data frame
  pred_nsq_floral_df <- cbind(nsq_data_floral,
                             cond_fit_log = pred_nsq_floral_cond$fit,
                             cond_se_log = pred_nsq_floral_cond$se.fit)

  #calculate predictions and confidence intervals on response scale (not log scale)
  pred_nsq_floral_df <- pred_nsq_floral_df %>%
    mutate(
      cond_fit = exp(cond_fit_log),
      cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
      cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
    )

  #plot
  ggplot(pred_nsq_floral_df,
         aes(x = bombus_floral_abun_scaled,
             y = cond_fit)) +
    geom_jitter(
      data = subset(surveys, cover_type == "herbaceous" & bee_season_type == "nestsearching" & plot_type == "ditch"),
      aes(x = bombus_floral_abun_scaled, y = nest_searching_queens),
      width = 0.05,
      height = 0,
      size = 3,
      alpha = 0.2,
      inherit.aes = FALSE) +
    geom_line(linewidth = 1.5) +
    geom_ribbon(aes(ymin = cond_lower, ymax = cond_upper),
              alpha = 0.2,
              color = NA) +
    labs(
      x = "Floral abundance (scaled)",
      y = "Predicted mean\nNSQ count\n(per plot)"
    ) +
    theme_classic(base_size = 30)

```



Floral abundance x plot type

```
#create new data grid
nsq_data_floral<- expand.grid(
  cover_type = "herbaceous",
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = floral_seq,
  bee_season_type = "nestsearching"
)

# Get predicted probabilities and standard errors
pred_nsq_floral <- predict(nsq_glm_veg_full,
  newdata = nsq_data_floral,
  se.fit = TRUE,
  re.form = NA,
  type = "link")

# Combine predictions with data frame
pred_nsq_floral_df <- cbind(nsq_data_floral,
  fit_log = pred_nsq_floral$fit,
  se_log = pred_nsq_floral$se.fit)

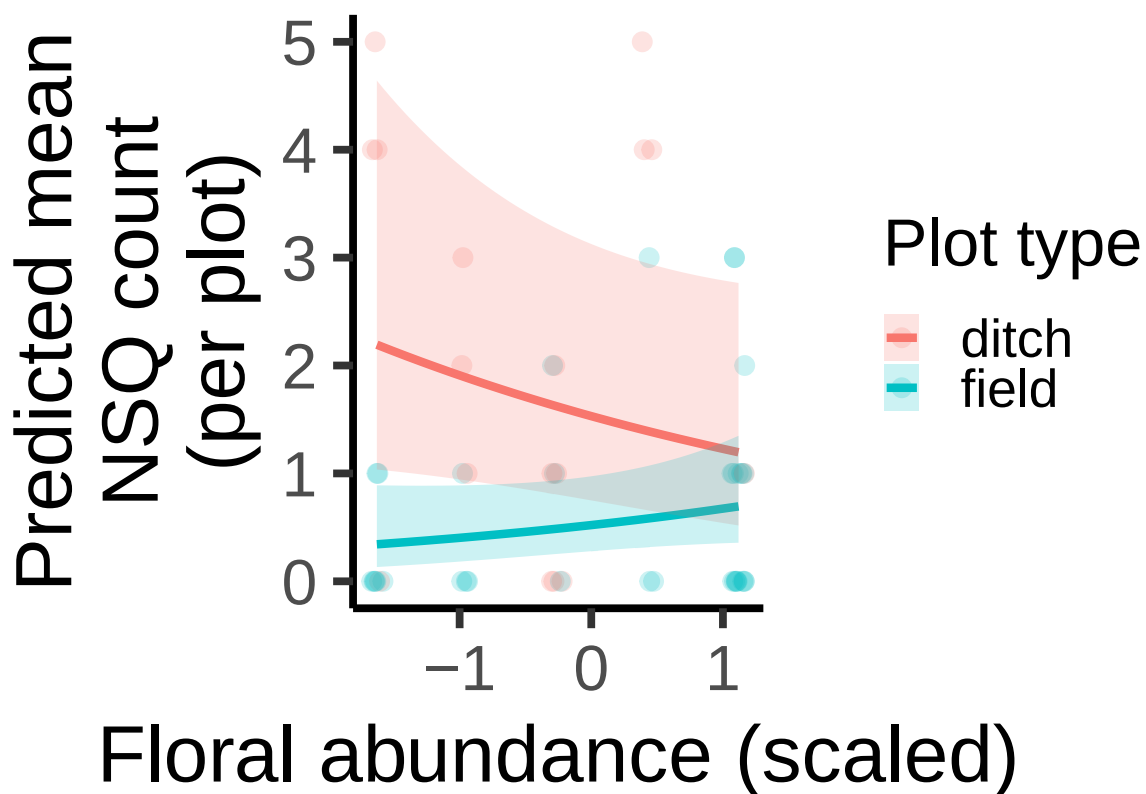
#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_floral_df <- pred_nsq_floral_df %>%
  mutate(
    fit = exp(fit_log),
```

```

    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#Plot
ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = fit, color = plot_type)) +
  geom_jitter(
    data = subset(surveys, cover_type == "herbaceous" & bee_season_type == "nestsearching"),
    aes(x = bombus_floral_abun_scaled, y = nest_searching_queens, color = plot_type),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = lower, ymax = upper, fill = plot_type), alpha = 0.2, color=NA) +
  labs(x = "Floral abundance (scaled)",
       y = "Predicted mean\nNSQ count\n(per plot)",
       fill = "Plot type",
       color = "Plot type") +
  theme_classic(base_size = 30) +
  theme(
    legend.title = element_text(size = 25),
    legend.text = element_text(size = 20),
    legend.key.size = unit(0.5, "cm")
  )

```



Season

```
# Create newdata with discrete factor levels for bee_season_type
nsq_data_season <- data.frame(
  cover_type = "herbaceous",
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled),
  bee_season_type = factor(c("nestsearching", "worker"), levels = levels(surveys$bee_season_type)),
  plot_type = "ditch"
)

# Get predicted zero probabilities and standard errors
pred_nsq_season_cond <- predict(nsq_glm_veg_full,
  newdata = nsq_data_season,
  type = "link",
  se.fit = TRUE,
  re.form = NA)

# Combine predictions with data frame
pred_nsq_season_df <- cbind(nsq_data_season,
  cond_fit_log = pred_nsq_season_cond$fit,
  cond_se_log = pred_nsq_season_cond$se.fit)

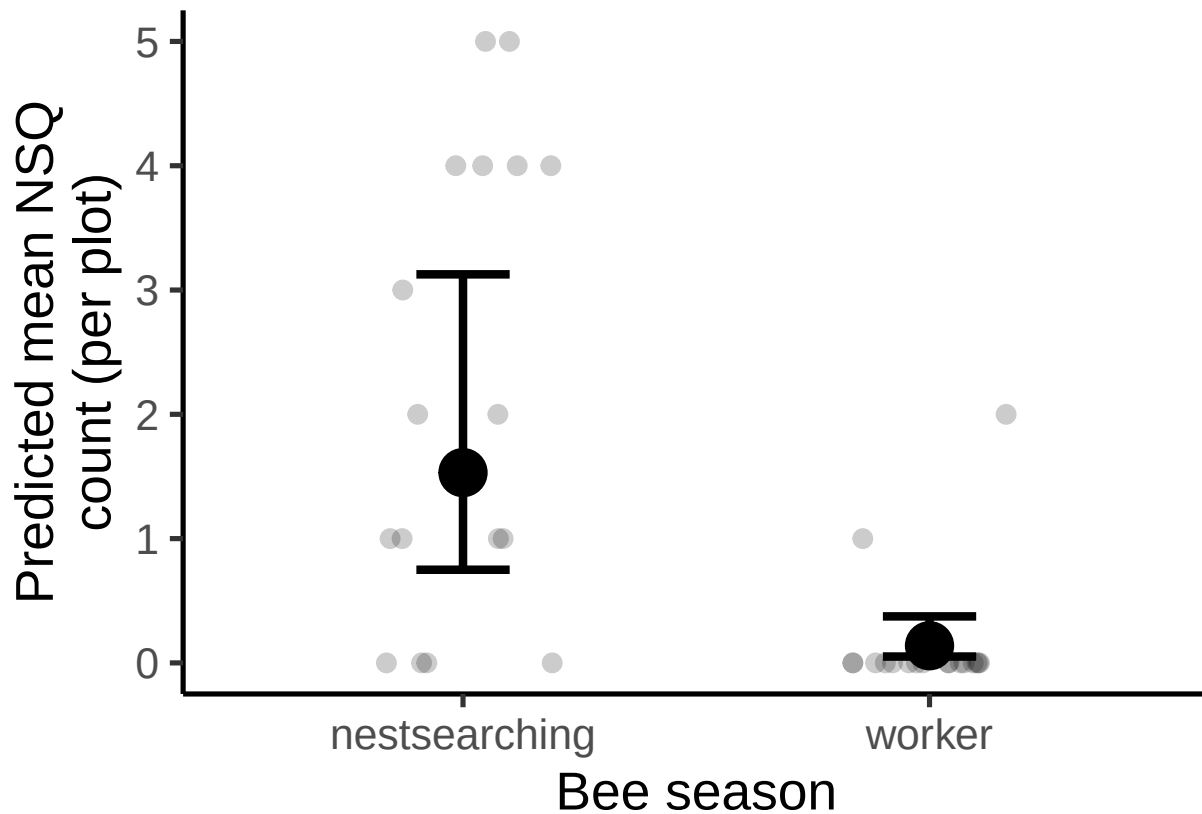
# calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_season_df <- pred_nsq_season_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
```

```

cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
)

ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = cond_fit)) +
  geom_jitter(
    data = subset(surveys, cover_type == "herbaceous" & plot_type == "ditch"),
    aes(x = bee_season_type, y = nest_searching_queens),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
    labs(x = "Bee season",
      y = "Predicted mean NSQ\ncount (per plot)" ) +
  theme_classic(base_size = 20)

```



Findings

Conditional model (predicts abundance of nest-searching queens at sites where queens are present). Ditch is reference plot type.

- No significant effect of cover type, plot type, floral abundance, or season on NSQ abundance

- No significant interaction between cover type and plot type of floral abundance and plot type

Zero-inflation model (predicts presence/absence of nest-searching queens).

- NSQ are more likely to be absent from bare sites than grassy sites
- NSQ are more likely to be absent during worker season than NSQ season
- No significant effect of plot type or floral abundance on NSQ abundance
- No significant interaction between cover type and plot type of floral abundance and plot type

Workers

Steps

- 1) Run poisson model with full RE

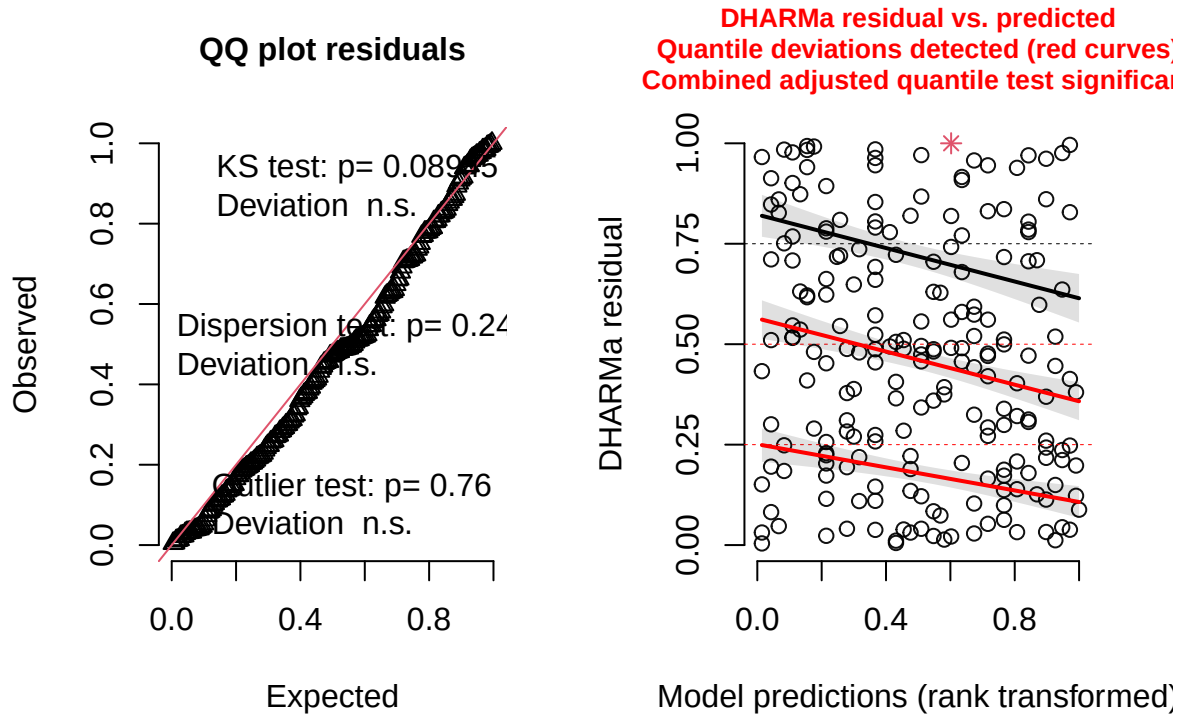
```
workers_veg_full <- glmmTMB(workers ~ cover_type*plot_type +
                             bombus_floral_abun_scaled*plot_type +
                             bee_season_type+
                             (1|area/site_id/plot_id),
                             family = poisson,
                             data = surveys)
```

- 2) Check diagnostics

- looks okay

```
residuals_workers_veg_full <- simulateResiduals(fittedModel = workers_veg_full, plot = TRUE)
```

DHARMA residual

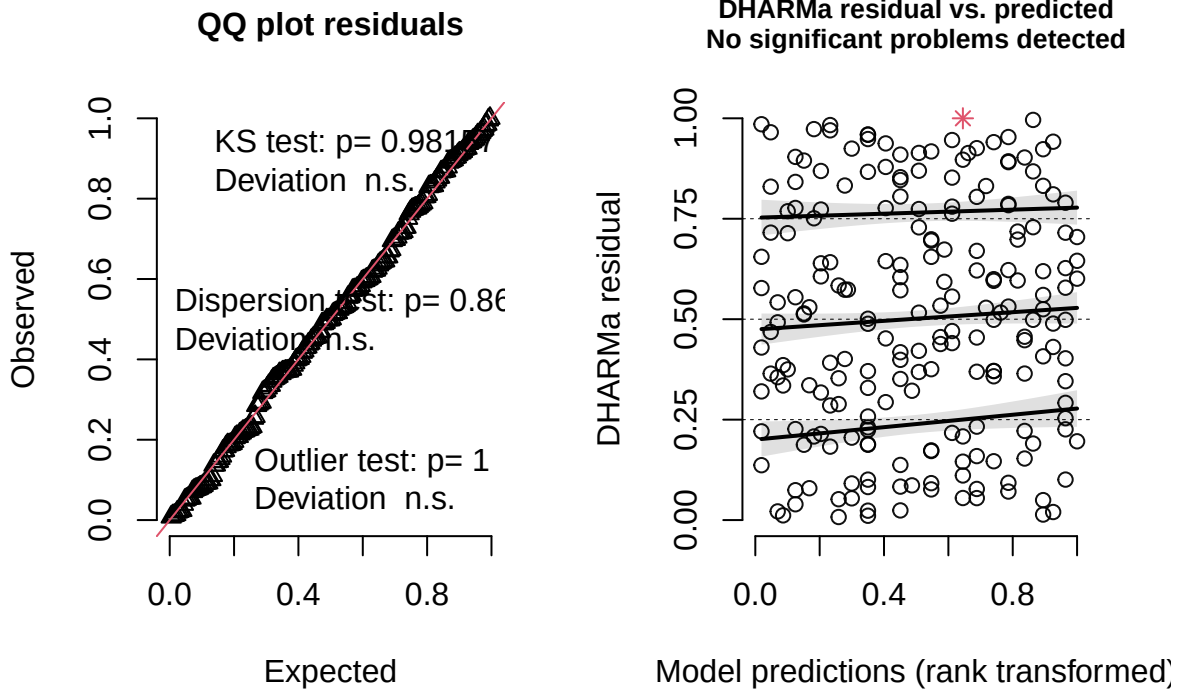


3. Try negative binomial

```
workers_veg_full_nbin <- glmmTMB(workers ~ cover_type*plot_type +
  bombus_floral_abun_scaled*plot_type +
  bee_season_type+
  (1|area/site_id/plot_id),
  family = nbinom2(),
  data = surveys)

residuals_workers_veg_full_nbin <- simulateResiduals(fittedModel = workers_veg_full_nbin, plot = TRUE)
```

DHARMA residual



##looks good!

3. Plot results.

```
summary(workers_veg_full_nbin)
```

```
## Family: nbinom2 ( log )
## Formula:
## workers ~ cover_type * plot_type + bombus_floral_abun_scaled *
##          plot_type + bee_season_type + (1 | area/site_id/plot_id)
## Data: surveys
##
##          AIC          BIC    logLik -2*log(L)  df.resid
##          269.3         306.1    -123.6    247.3      199
##
## Random effects:
##
## Conditional model:
## Groups              Name          Variance Std.Dev.
## plot_id:site_id:area (Intercept) 3.889e-09 6.236e-05
## site_id:area         (Intercept) 7.871e-02 2.805e-01
## area                 (Intercept) 1.154e-01 3.398e-01
## Number of obs: 210, groups:
## plot_id:site_id:area, 36; site_id:area, 12; area, 6
##
```

```
## Dispersion parameter for nbinom2 family (): 0.229
##
## Conditional model:
##
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -2.5300     0.7217  -3.506 0.000456
## cover_typeherbaceous -0.4157     0.7389  -0.563 0.573648
## plot_typefield     -0.7885     0.6167  -1.279 0.201016
## bombus_floral_abun_scaled 0.2428     0.5478   0.443 0.657511
## bee_season_typeworker 2.4446     0.5789   4.223 2.41e-05
## cover_typeherbaceous:plot_typefield 0.3604     0.9140   0.394 0.693315
## plot_typefield:bombus_floral_abun_scaled 0.5713     0.6750   0.846 0.397332
##
## (Intercept)          ***
## cover_typeherbaceous
## plot_typefield
## bombus_floral_abun_scaled
## bee_season_typeworker          ***
## cover_typeherbaceous:plot_typefield
## plot_typefield:bombus_floral_abun_scaled
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Plots

Plot type

```
# Create newdata
worker_data_plotttype <- expand.grid(
  cover_type = "herbaceous",
  plot_type = factor(c("ditch", "field"), levels = levels(surveys$plot_type))
)
worker_data_plotttype$bombus_floral_abun_scaled <- mean(surveys$bombus_floral_abun_scaled)
worker_data_plotttype$bee_season_type <- "worker"

# Get predicted probabilities and standard errors
pred_worker_plotttype <- predict(workers_veg_full_nbin,
  newdata = worker_data_plotttype,
  se.fit = TRUE,
  re.form = NA,
  type = "link")

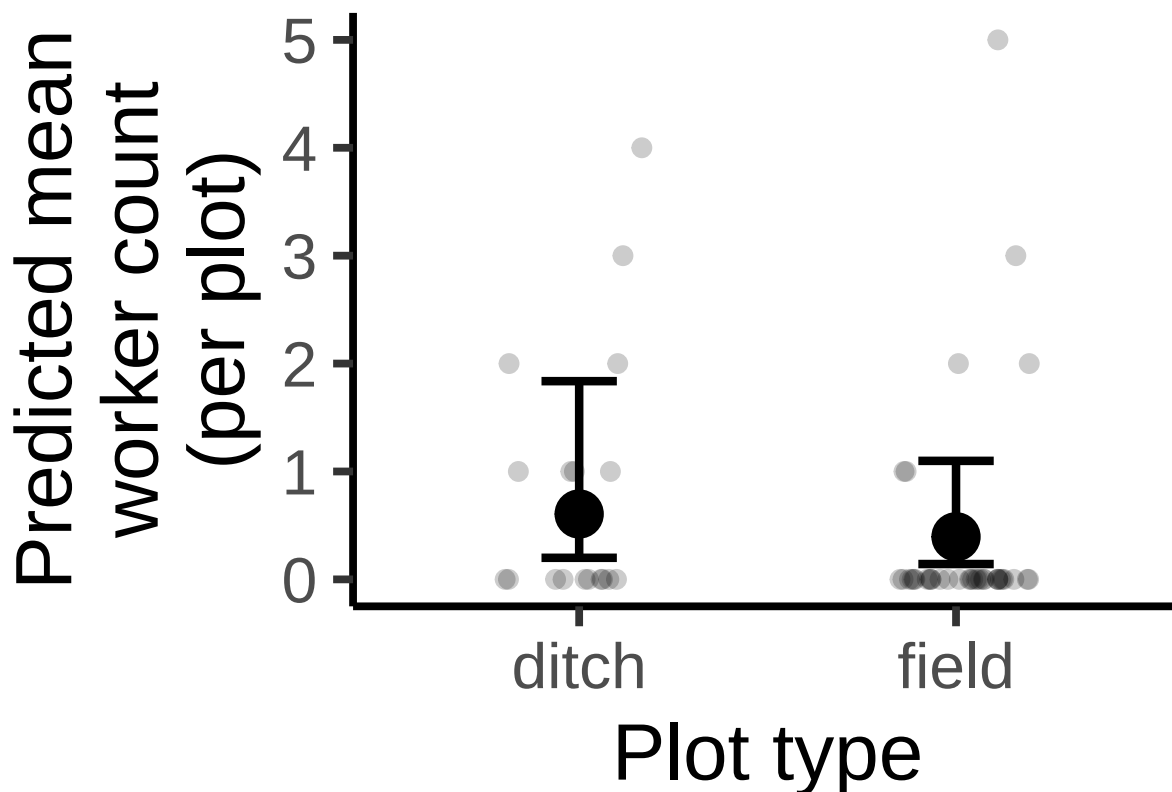
# Combine predictions with data frame
pred_worker_plotttype_df <- cbind(worker_data_plotttype,
  fit_log = pred_worker_plotttype$fit,
  se_log = pred_worker_plotttype$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_plotttype_df <- pred_worker_plotttype_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )
```

```

#Plot
ggplot(pred_worker_plottype_df, aes(x = plot_type, y = fit)) +
  geom_jitter(
    data = subset(surveys, cover_type == "herbaceous" & bee_season_type == "worker"),
    aes(x = plot_type, y = workers),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth = 1) +
  labs(x = "Plot type",
       y = "Predicted mean\nworker count\n(per plot)") +
  theme_classic(base_size = 30)

```



Cover type

```

# Create newdata
worker_data_covertype <- expand_grid(
  cover_type = factor(c("herbaceous", "bare"), levels = levels(surveys$cover_type)),
  plot_type = "ditch"
)
worker_data_covertype$bombus_floral_abun_scaled <- mean(surveys$bombus_floral_abun_scaled)
worker_data_covertype$bee_season_type <- "worker"

```

```

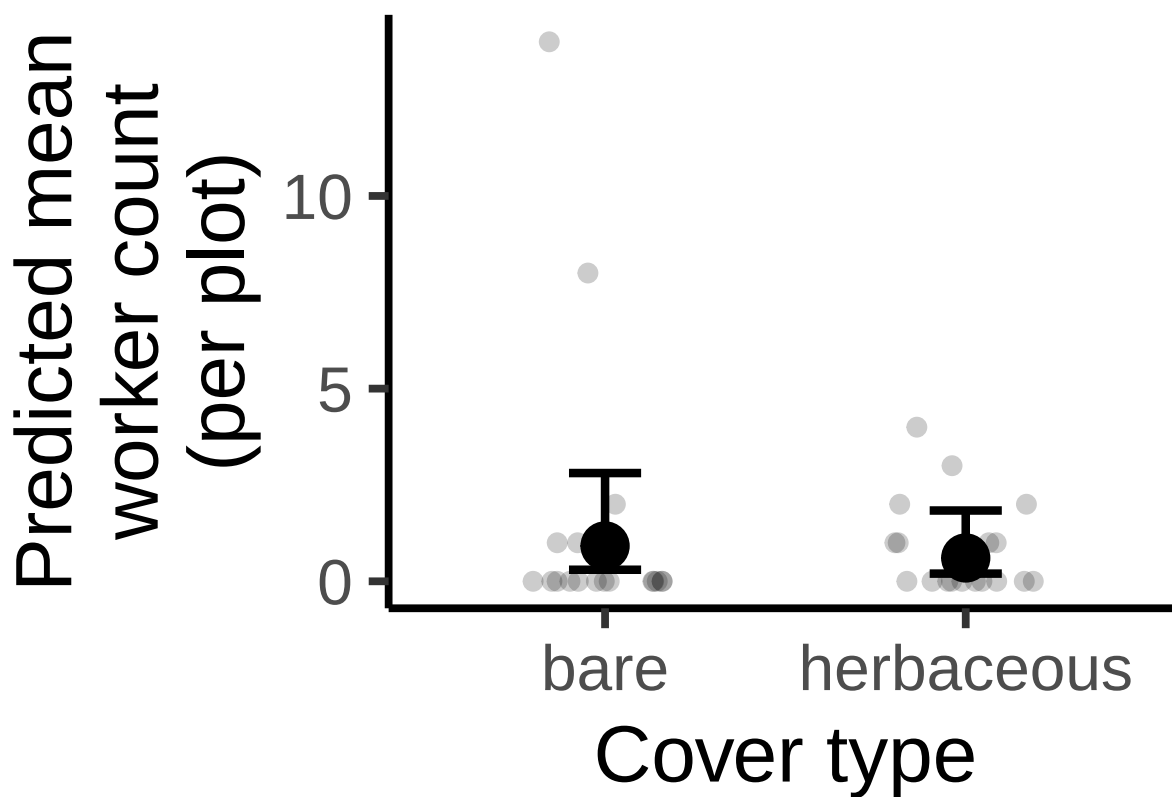
# Get predicted probabilities and standard errors
pred_worker_covertime <- predict(workers_veg_full_nbin,
                                newdata = worker_data_covertime,
                                se.fit = TRUE,
                                re.form = NA,
                                type = "link")

# Combine predictions with data frame
pred_worker_covertime_df <- cbind(worker_data_covertime,
                                  fit_log = pred_worker_covertime$fit,
                                  se_log = pred_worker_covertime$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_covertime_df <- pred_worker_covertime_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#Plot
ggplot(pred_worker_covertime_df, aes(x = cover_type, y = fit)) +
  geom_jitter(
    data = subset(surveys, plot_type == "ditch" & bee_season_type == "worker"),
    aes(x = cover_type, y = workers),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidthh
  labs(x = "Cover type",
       y = "Predicted mean\nworker count\n(per plot)") +
  theme_classic(base_size = 30)

```



Floral abundance x plot type

```
#create new data grid
worker_data_floral<- expand.grid(
  cover_type = "herbaceous",
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = floral_seq,
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_floral <- predict(workers_veg_full_nbin,
                             newdata = worker_data_floral,
                             se.fit = TRUE,
                             re.form = NA,
                             type = "link")

# Combine predictions with data frame
pred_worker_floral_df <- cbind(worker_data_floral,
                               fit_log = pred_worker_floral$fit,
                               se_log = pred_worker_floral$se.fit)

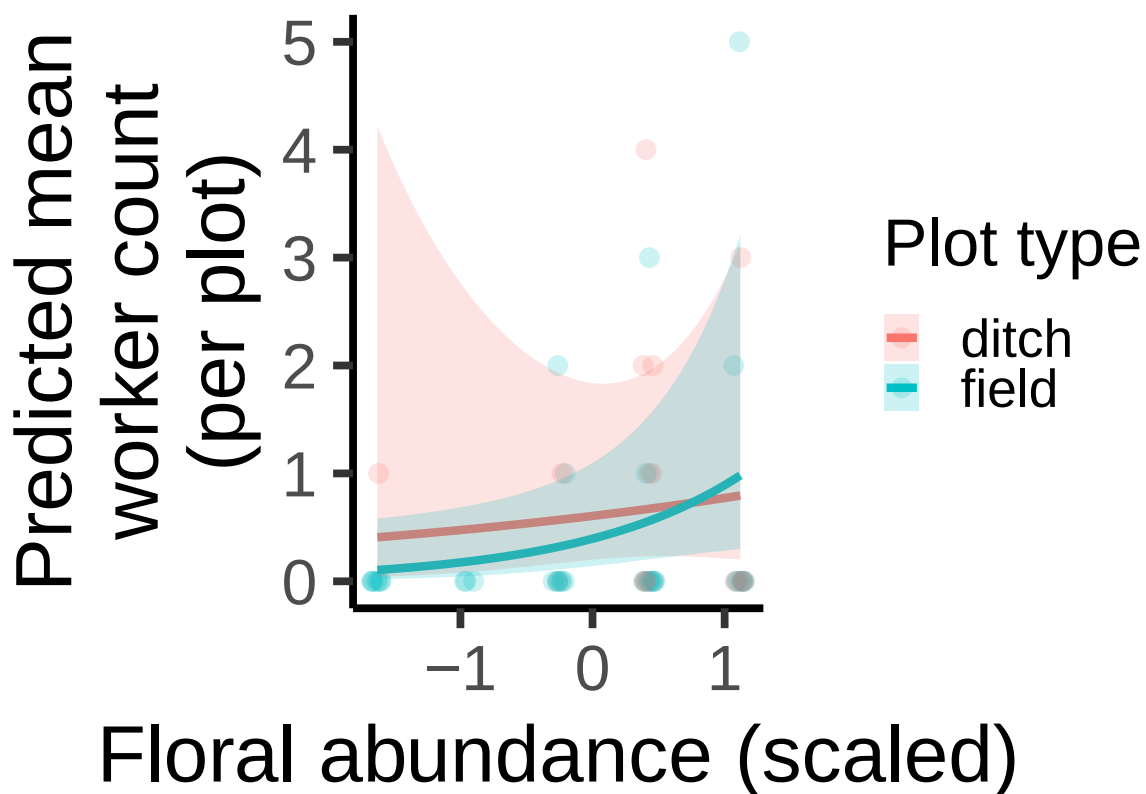
#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_floral_df <- pred_worker_floral_df %>%
  mutate(
    fit = exp(fit_log),
```

```

    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
)

#Plot
ggplot(pred_worker_floral_df, aes(x = bombus_floral_abun_scaled, y = fit, color = plot_type)) +
  geom_jitter(
    data = subset(surveys, cover_type == "herbaceous" & bee_season_type == "worker"),
    aes(x = bombus_floral_abun_scaled, y = workers, color = plot_type),
    width = 0.05,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = lower, ymax = upper, fill = plot_type), alpha = 0.2, color=NA) +
  labs(x = "Floral abundance (scaled)",
       y = "Predicted mean\nworker count\n(per plot)",
       fill = "Plot type",
       color = "Plot type") +
  theme_classic(base_size = 30) +
  theme(
    legend.title = element_text(size = 25),
    legend.text = element_text(size = 20),
    legend.key.size = unit(0.5, "cm")
  )

```



Season

```
#create new data grid
worker_data_season<- expand.grid(
  cover_type = "herbaceous",
  plot_type = "ditch",
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  bee_season_type = unique(surveys$bee_season_type)
)

# Get predicted probabilities and standard errors
pred_worker_season <- predict(workers_veg_full_nbin,
  newdata = worker_data_season,
  se.fit = TRUE,
  re.form = NA,
  type = "link")

# Combine predictions with data frame
pred_worker_season_df <- cbind(worker_data_season,
  fit_log = pred_worker_season$fit,
  se_log = pred_worker_season$se.fit)

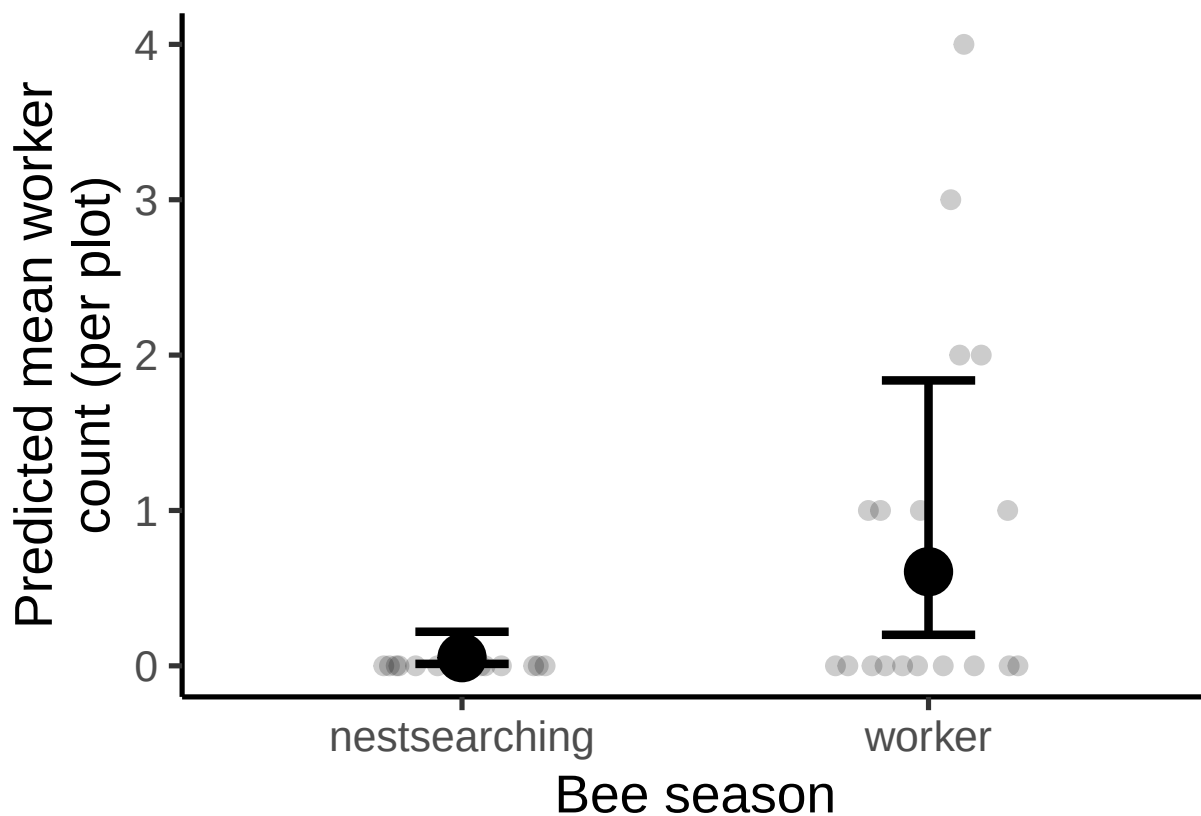
#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_season_df <- pred_worker_season_df %>%
  mutate(
    fit = exp(fit_log),
```

```

    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#Plot
ggplot(pred_worker_season_df, aes(x = bee_season_type, y = fit)) +
  geom_jitter(
    data = subset(surveys, plot_type == "ditch" & cover_type == "herbaceous"),
    aes(x = bee_season_type, y = workers),
    width = 0.2,
    height = 0,
    size = 3,
    alpha = 0.2,
    inherit.aes = FALSE) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth = 1) +
  labs(x = "Bee season",
       y = "Predicted mean worker\ncount (per plot)",
       fill = "Plot type",
       color = "Plot type") +
  theme_classic(base_size = 20)

```



Findings

Cover type

- No effect of cover type on worker abundance ($p=0.80$)
- More workers along the ditch than within the field ($p=0.0243$)
- No interaction between cover type and plot type

Floral abundance

- No overall impact of floral abundance on workers ($p=0.9984$)
- Significant interaction between plot type and floral abundance ($p=0.0169$)
- Floral abundance influences worker abundance within the field but NOT along the ditch

Date

- Significantly more workers in worker season than NS ($p<0.0001$)